How to Build a Scientific Scaling Ladder

This article is compiled strictly from **public literature and technical reports, containing no confidential material**. It lays out how to build a Scaling Ladder—using a structured matrix of small-scale training runs to fit scaling laws and extrapolate model architecture, hyperparameters, final loss, and downstream performance at target scale, de-risking flagship training runs before they launch.

1. Scope and Terminology

1.1 Definition of Scaling Ladder

A Scaling Ladder is a structured grid of small-scale training runs spanning parameter counts N, token horizons D, and other controlled variables. Its purpose is to fit empirical scaling laws within a manageable compute budget and quantitatively extrapolate optimal model size, training duration, hyperparameters, and expected performance at target scale—replacing trial-and-error at flagship scale with predictive engineering.

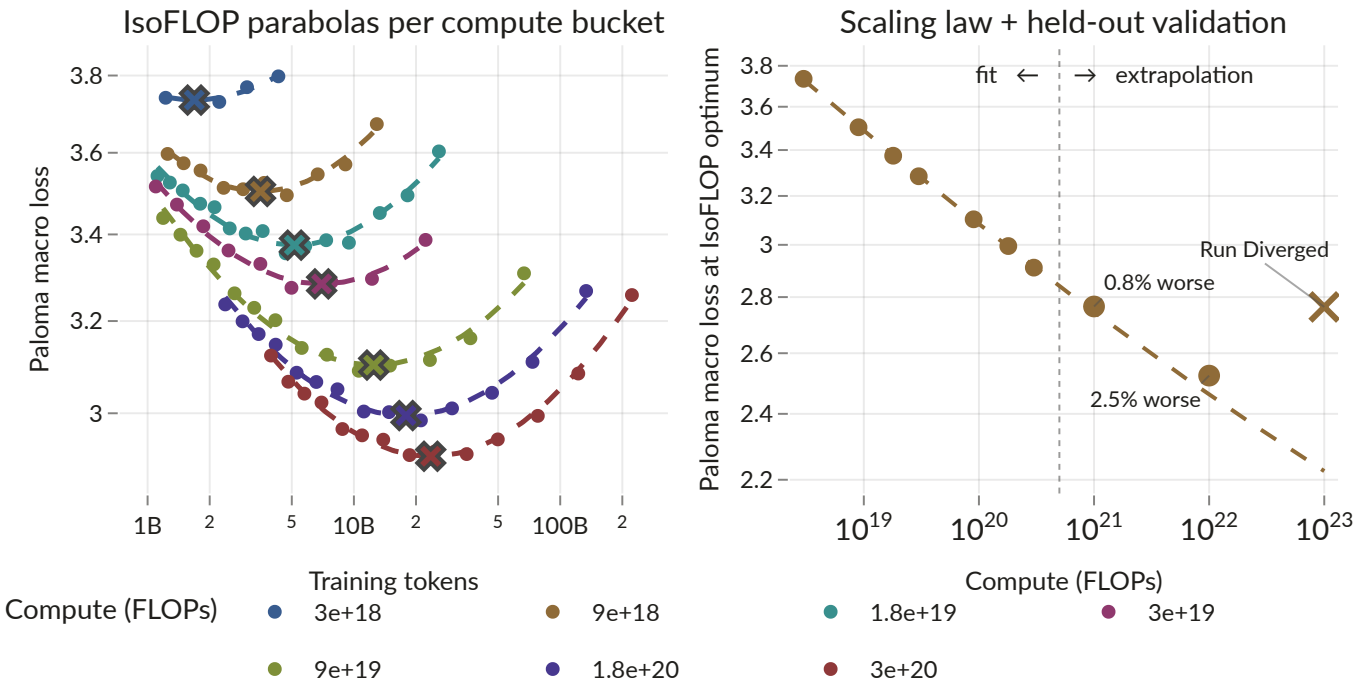


Figure 1. The first Delphi experiment (Cautious AdamC recipe). The large-scale training run in the right panel deviates from the prediction and diverges. Source: Marin, 2026.

A clean fit at small scale does not guarantee reliable extrapolation at large scale. As shown in Figure 1, Delphi's initial recipe tracked power-law predictions smoothly across the fitting regime, yet drifted 2.5% above predicted loss at 10^{22} FLOPs and diverged outright at 10^{23} FLOPs. A scientific Ladder therefore requires not only fitting points, but also out-of-sample holdouts and stability stress tests before signing off on a target run (§10).

1.2 Typical Scenarios

We focus on three pretraining regimes. Chapters 2–10 cover the shared workflow, while considerations specific to MoE and data-constrained training are addressed in dedicated subsections within each chapter.

Scenario	Core Challenges	Dedicated Sections
Dense	Aspect-ratio scaling rules and compute allocation; used both for dense models and as a controlled baseline for MoE	§4.3
MoE	Decoupled total vs. active parameters; sparsity, expert granularity, routing, and load balancing; realized throughput under expert parallelism	§3.1, §4.4, §5.6, §6.6, §8.5, §10.3
Data-constrained	Available unique tokens fewer than target training tokens; epoch repetition, data quality, and domain mixture jointly bound achievable loss	§5.3, §7.3

In practice, MoE and data constraints frequently overlap. Driven by inference budgets or finite high-quality corpora, the target TPP (tokens per parameter) often sits well above or below the Chinchilla compute-optimal ratio, so the token horizon axis must be centered around the target TPP (§5.3).

Scaling laws for post-training (SFT/RL) itself, native multimodality, and distillation or synthetic data generation pipelines are outside our scope; post-training enters only as a downstream plasticity check on pretraining candidates (§2.2).

1.3 Outputs of the Ladder

Output	Extrapolated Target	Section
Hyperparameter scaling law (LR, BSZ, WD)	Optimal training hyperparameters at target scale	§6
Loss scaling law	Final training / validation loss after annealing	§8.1
Loss curve scaling law	Full training trajectory and annealing gain	§8.2
Annealing ratio	Fraction of total tokens allocated to LR decay	§6.5 (empirical starting range only)
Data mixture and data repetition scaling law	Optimal domain weights; effective token count under multi-epoch training	§7
MoE sparsity scaling law	Total expert count and granularity at fixed active parameters	§5.6, §8.5
Downstream task scaling law	Benchmark performance at target scale	§9

1.4 Terminology and Notation

Term	Definition
N_{body}	Transformer trunk parameter count, excluding input embedding and output head (§3.1)
N_{total}, N_{active}	MoE trunk total parameters (across all experts) and active parameters per token
D	Cumulative training tokens contributing to the loss (§3.3)
U	Deduplicated unique tokens available
C	Total training FLOPs, counted by actual architecture and kernel execution (§3.2)
TPP	Tokens per parameter (D/N); always align the convention of N when comparing across papers
LR (η), BSZ (B), WD (λ)	Learning rate, batch size (measured in tokens), and weight decay
Holdout	Out-of-sample large-scale runs excluded from curve fitting and candidate selection, reserved strictly for extrapolation checks
Fully-Tuned Frontier	The lower-envelope loss achieved when hyperparameters are thoroughly tuned at each grid point (§6.1)
Equivalent compute multiplier	Ratio of compute required to reach the same loss, providing a scale-invariant measure of performance gaps (§2.3)

2. Decision Objectives and Acceptance Criteria

2.1 Decision Objectives

Different engineering goals demand very different sampling grids and tuning depths, so the objective must be locked down before running experiments:

1. Fixed-recipe extrapolation: predicting how a prescribed hyperparameter scaling rule performs at target scale, without exhaustive per-point grid search.
2. Optimal compute allocation: solving for the loss-minimizing (N, D) split under a fixed compute budget C.
3. Candidate comparison: testing whether a new architecture, optimizer, or data mixture beats the baseline at target scale.
4. Cross-scale hyperparameter prediction: fitting how (η, B, λ) scale with N and D.

Delphi (Marin, 2026) highlights an essential distinction: a recipe's competitiveness (achieving low loss) and its predictability (extrapolating accurately from small to large scale) are orthogonal properties. Objectives 2–4 require every fitting point to be tuned onto the Fully-Tuned Frontier (§6.1); if small models are under-tuned, their inflated losses tilt the fitted power-law slope. Objective 1 simply executes the recipe's pre-specified scaling formulas.

2.2 Evaluation Protocol

Multiple non-linear transformations separate pretraining cross-entropy from final model capabilities, creating several evaluation traps:

- Decoupling between proxy loss and downstream accuracy: small-scale training loss improvements do not automatically translate into target-scale benchmark gains (Qwen3.8-Next, 2026 §1).
- Poor signal-to-noise at small scales: on reasoning-heavy tasks like math and code, small models score near random guessing on discrete accuracy, whereas easy tasks saturate at large scales. Where accuracy is flat at small scale, substitute continuous proxies such as answer bits-per-byte (BPB) (OLMo 3, 2025).
- Cross-scale rank inversions: high small-scale discriminability only proves low noise; using a proxy to select candidates requires verifying rank correlation between small-scale proxy scores and target-scale capability metrics (OLMo 3, 2025 §3.3, Appendix A.4).
- High-variance benchmark noise: evaluation suites should cover core target capabilities while multi-sampling or isolating noisy benchmarks so their variance does not mask genuine trends (Phi-4, 2024 §5; OLMo 3, 2025 §3.3.3–3.3.4).
- Evaluation version lock: freeze prompt templates, decoding parameters, and scoring scripts, and run strict decontamination checks between training corpora and evaluation sets.

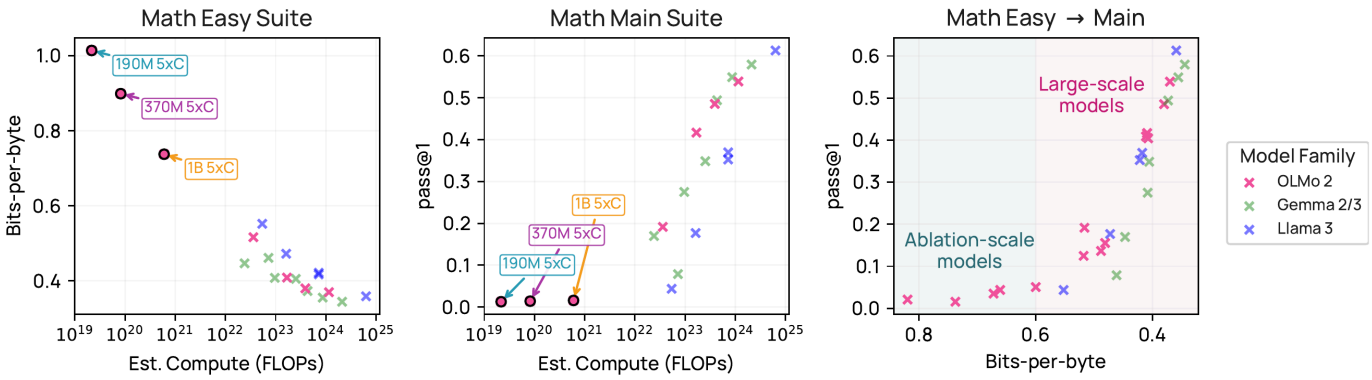


Figure 2. Math evaluation of OLMo 3: the left and middle panels show bits-per-byte (BPB) on the Easy suite and pass@1 on the Main suite as functions of compute, respectively; small-scale models already show distinguishable differences in BPB, while pass@1 is near zero over the same period. The right panel shows the relationship between the two types of metrics across the models examined; ranking across scales still needs to be validated for the target task. Source: Olmo 3, 2025, v1, Fig. 6 (PDF page 12).

When the final deliverable is a post-trained model (SFT/RL), evaluating pretraining checkpoints alone can miss latent defects that surface only during post-training. Qwen3.8-Next, 2026 documents two striking examples: removing positional embeddings (NoPE) and adding a sparse-read residual branch each had negligible impact on pretraining loss, yet NoPE caused runaway non-terminating generation after post-training and the sparse-read branch degraded post-training quality. Consequently, OLMo 3, 2025 §3.5.1 subjects fully annealed candidate mixtures to fast instruction tuning, checking both capability scores and generation termination and length.

2.3 Acceptance Thresholds and Decision Rules

Validation of a Ladder centers on four questions:

1. Point prediction error of a fixed recipe at target compute;
2. Predicted margin and ranking confidence between competing candidates at target scale;
3. Expected loss regret of the chosen (N, D) allocation relative to neighboring ratios;
4. Numerical stability and capability floors at scale.

Acceptance thresholds must be fixed before inspecting holdout runs, calibrated against random seed noise and the minimum meaningful effect size. Choshen et al., 2025 report that modeling changes adopted in the literature typically require ~4% relative improvement, while random seed restarts alone fluctuate by ~3.5%; Delphi's 0.2%–0.5% prediction error reflects a specific low-noise regime rather than a universal pass/fail bar.

Relative loss error is also a poor cross-scale metric. Under a power law $L(C) = E + AC^{-\gamma}$, a small loss delta ΔL translates into an equivalent compute multiplier of:

$$\ln \frac{C_2}{C_1} \approx \frac{\Delta L}{\gamma\,(L - E)}$$

As scale increases and L approaches the irreducible entropy E, the same ΔL represents a dramatically larger compute ratio. Defining acceptance bars in terms of equivalent compute multipliers (e.g., a 15% compute savings) alongside absolute ΔL avoids scale distortion.

When comparing two candidates across the same ladder, shared sampling and evaluation noise partially cancels—so confidence intervals should be estimated directly on the paired difference ΔL rather than by adding independent error bars. Conversely, a low average loss error does not guarantee a stable (N_{opt}, D_{opt}) optimum: Gemstones, 2025 §4.2 demonstrates that varying which grid points are included in the fit can noticeably shift compute-allocation recommendations. Fit uncertainty must therefore be propagated all the way to the target decision variable (candidate gap or optimal N).

3. Measurement Specification

3.1 Parameter Count Definition

The parameter count N used to fit $L(N, D)$ and the parameter count used to tally FLOPs C serve different roles and should be tracked separately.

When fitting $L(N, D)$, use the Transformer trunk parameter count N_{body}, excluding input embeddings and the output head. Embedding and head parameters scale as $O(Vd)$ whereas the trunk scales as $O(d^2 n)$; at small scales, vocabulary parameters represent a large fraction of the total and distort power-law exponents if included. Farseer (Li et al., 2025) Appendix G confirms that including embeddings degrades extrapolation accuracy at 25.1B, consistent with Kaplan et al., 2020.

When computing training FLOPs C, however, the output head executes a dense $d \times V$ matrix multiplication on every token. Even when weights are tied with the input embedding, those output projection FLOPs are physically executed and must be counted in C, whereas the input embedding lookup is a memory-bandwidth operation logged separately.

Always check parameter conventions when citing external numbers: Porian et al., 2024 Table 2 includes the output head in N while excluding the input embedding, whereas Chinchilla's well-known 20 TPP ratio counts total parameters including all embeddings (Hoffmann et al., 2022 Appendix F).

For MoE models, each grid point should simultaneously record:

- Trunk total parameters N_{total} and per-token active parameters N_{active} (with embedding and head logged separately);
- Total routed experts E_{total}, active routed experts per token E_{active}, and sparsity $S = E_{total}/E_{active}$;
- Count and size of shared experts, plus individual routed expert size (expert granularity);
- Router architecture and load-balancing configuration (§4.4).

3.2 Compute

The textbook approximation $C \approx 6ND$ omits quadratic attention and vocabulary projection FLOPs. For standard MHA and a $4d$ FFN evaluated over the full $L \times L$ attention matrix, forward-plus-backward compute is:

$$C \approx \left(6 + \frac{L}{d}\right) N_\text{body} D + 6VdD$$

where L is sequence length, d is hidden dimension, V is vocabulary size, and $N_{\text{body}} \approx 12d^2n$ across n layers. The L/d term captures context attention and the final term captures the output head. If the causal FlashAttention kernel skips masked upper-triangle tiles, the attention term is roughly halved; SwiGLU, GQA, and MoE further modify the constants, so C should always be tallied from the actual architecture and kernel execution.

For MoE models, matrix-multiplication FLOPs scale with N_{active} plus router overhead; expert-parallel (EP) all-to-all communication adds zero FLOPs but directly impacts wall-clock training time and must be accounted for in realized efficiency (§10.5).

3.3 Loss and Token Conventions

To keep runs comparable across scales and batches, lock down the following measurement conventions across the entire Ladder:

Dimension	Convention Requirement
Primary loss	Pure next-token cross-entropy; log MoE auxiliary losses, z-loss, and MTP heads separately without mixing them into the fitted loss
Evaluation distribution	Evaluate all candidates on an identical held-out corpus and domain weighting, reporting key domains separately
Loss aggregation	Specify per-token vs. per-document averaging and standardize the distributed reduction denominator
Sequence packing & masks	Standardize BOS/EOS insertion, document boundary masks, packing, position IDs, and truncation
Token accounting	Distinguish total processed tokens, loss-bearing tokens, deduplicated unique tokens, and epoch exposures
Random seeds	Log initialization, data sampling, and shuffle order seeds separately, using paired seeds for candidate ablations

Dimension	Convention Requirement
Cross-tokenizer comparison	Convert losses to byte-normalized metrics (such as BPB) and compare wall-clock compute on identical raw text

When experiments vary the training data mixture, training loss on each run's own distribution reflects shifts in corpus entropy (e.g., higher code fractions naturally yield lower cross-entropy) and cannot rank model quality; candidates must be compared on a fixed validation distribution.

Similarly, trigger validation evaluations at a fixed fraction of total steps T (e.g., every 2% of training progress) rather than at a fixed step interval (e.g., every 1,000 steps). Fixed step intervals cause longer runs to generate far more evaluation points, over-weighting long trajectories in curve fits (§8.3).

4. Baseline Recipe and Scaling Rules

4.1 Variable Classification

Every configuration knob in a Ladder should be assigned to one of three roles to prevent confounding variables:

Variable Category	Role	Typical Examples
Fixed invariants	Held constant across the entire Ladder	Model family, dataset version, optimizer type, sequence length
Prescribed scaling rules	Co-varied with N or D by a deterministic formula	LR (power law or μP), BSZ ($D^{0.4}$ prior), warmup fraction
Independent variables	Swept on the grid or solved via fitting	N, D, and optimal tuned LR and BSZ

A classic pitfall is fixing the warmup step count across scales. A constant warmup step count consumes a disproportionately large fraction of short, small-budget runs, systematically biasing fitted compute-optimal exponents (Porian et al., 2024). Warmup should instead scale as a fixed fraction of total training tokens—for instance, Delphi uses 10% of total training volume (Appendix A.3).

Whenever the base architecture, optimizer, dataset version, numerical precision, or tokenizer changes, existing scaling exponents can shift; test small-scale transferability first before choosing between local recalibration and a full Ladder rerun (§12.3).

4.2 Architecture and Training Configuration Consistency

Within a model family, all architectural primitives other than the explicit experimental variable must remain invariant across scales:

Architectural Property	Consistency Requirement
Backbone topology	Uniform architecture (e.g., decoder-only Transformer across all sizes)
Normalization	Identical placement and type (e.g., Pre-RMSNorm)
Positional encoding	Identical scheme and base frequency (e.g., RoPE)
Activation function	Identical activation and FFN expansion ratio (e.g., SwiGLU)
Embedding tying	Consistently tied or consistently untied across all sizes
Attention mechanism	Uniform attention type and grouping rule (e.g., GQA)
Aspect-ratio progression	Scale width and depth along a single prescribed rule (§4.3)
MoE routing setup	Keep routing algorithm, load balancing, and shared-expert ratio consistent (§4.4)

Training environment and outer-loop settings must likewise be locked:

Training Configuration	Typical Setting
Sequence length	4,096
Training data version	Locked dataset snapshot and mixture version
Optimizer	AdamW
Validation set	Identical evaluation corpus (§3.3)
Evaluation frequency	Triggered at fixed percentages of total training steps
Infrastructure conventions	Identical tokenizer, initialization scheme, parameterization, and loss reduction

Small proxy models should mirror the target model's structural design—such as GQA group ratios, `head_dim`, and width-to-depth progression—so their memory-bandwidth bottlenecks and representational dynamics remain valid proxies for the target run.

4.3 Width-Depth Configuration

At fixed trunk parameter count N_{body}, varying the ratio of hidden width d to layer count n alters the attention FLOP share L/d (§3.2). While Kaplan et al., 2020 observed that pretraining loss is relatively flat across a broad range of aspect ratios at fixed N, recent controlled studies show that width-to-depth ratios meaningfully affect downstream benchmark performance (Gemstones, 2025 §4.4) and complex reasoning capabilities (GLM-4.5, 2025). Enforce a consistent width-depth scaling rule across the Ladder, and add targeted structural controls whenever depth or head dimensions are under evaluation.

4.4 MoE Structural Scaling Rules

Beyond the general invariants of §4.2, an MoE Ladder must standardize five structural dimensions:

- Expert granularity: Krajewski et al., 2024 treat expert size as an independent scaling dimension, demonstrating that the conventional choice of setting expert size equal to a standard dense FFN is suboptimal across nearly all compute budgets; finer-grained experts widen MoE's advantage over dense models as scale grows.
- Shared experts: fix the count of dedicated shared experts and their capacity ratio relative to routed experts.
- Load-balancing strategy: lock the auxiliary loss coefficient or the bias update rate in auxiliary-loss-free routing. DeepSeek-V3, 2024 §2.1.2 updates routing biases dynamically by expert load, setting the bias update speed to 0.001 for the first 14.3T tokens and 0 for the final 500B tokens. Because balancing strength directly trades modeling loss against EP communication throughput, altering it constitutes a recipe change.
- Capacity factor and token dropping: keep capacity factors consistent between training and inference and log token drop rates as a primary health metric; DeepSeek-V3 drops zero tokens in both training and inference (§2.1.2).
- Sparsity: ignoring memory and communication overhead, Abnar et al., 2025 show that at fixed training FLOPs, raising sparsity (and thus N_{total}) steadily lowers pretraining loss; at fixed N_{total}, loss follows a U-shaped parabola in sparsity whose optimum shifts higher with compute budget. On most downstream tasks, performance tracks pretraining loss regardless of sparsity, except on reading comprehension benchmarks (such as CoQA and SQuAD) where denser models retain an edge. Ludziejewski et al., 2025 jointly model expert count, N_{active}, and D under explicit memory constraints (tested up to 2.7B active / 5B total parameters).

4.5 Vocabulary and Numerical Precision

- Vocabulary size: Tao et al., 2024 show across 33M–3B models that optimal vocabulary size grows as a power law of compute, and that most open-weight models use undersized vocabularies. Because changing vocabulary size simultaneously alters $O(Vd)$ head FLOPs and characters-per-token compression, cross-vocabulary comparisons must use byte-normalized loss (§3.3).
- Numerical precision: Kumar et al., 2024 (validated up to 1.7B parameters and 26B tokens) fold numerical precision into scaling laws: low-precision training reduces effective parameter capacity, while heavier over-training (higher TPP)

amplifies loss degradation from post-training quantization. Run the Ladder under the exact target precision scheme, and re-verify implementation parity whenever precision settings change (§10.4).

5. Experimental Design and Budget

5.1 Experimental Matrix Structure

A typical (N, D) experimental grid takes the form:

```
Training budget D →  0.5×    1×     2×     4×
Model   130M         ●       ●      ●      ●
size    520M         ●       ●      ●      ●
        2.3B         ●       ●      ●      ●
N       8B           ○       ○      ○      ○  ← holdout (excluded from fitting)
↓
```

- Fitting points (●): smaller and medium sizes swept across token horizons to estimate Scaling Law parameters.
- Holdout points (○): largest sizes or higher-compute budgets excluded from fitting and reserved strictly to test extrapolation accuracy.

The geometry of the sampling grid must be co-designed with the functional form:

- Clarify whether the target decision requires extrapolating along N, along D, or jointly across (N, D), and annotate epoch repetition counts whenever data is constrained;
- Choose between a full Cartesian grid, IsoFLOP slices, or a compute-saving L-shape layout based on the decision objective in §2.1 (§8.1);
- Reserve validation points and contingency budget upfront to resolve functional-form divergence or high seed noise (§5.5).

Pay special attention to parameter identifiability: if every run in the Ladder is sampled along a single fixed TPP ray (e.g., $D = 20N$ everywhere), N and D are perfectly collinear and regression cannot disentangle their individual exponents. Holding out an entire size tier for cross-validation or inspecting the parameter covariance matrix catches degenerate grids before compute is wasted.

5.2 Number of Sizes, Span, and Extrapolation Multiplier

Design Dimension	Practical Guideline	Source
Number of model sizes	At least 3–4 fitting sizes plus dedicated holdouts; additional sizes stabilize slope estimates	Choshen et al., 2025
Size span	Span a wide range toward the target; power-law linearity holds across a 34× span on several model families	Choshen et al., 2025
Adjacent size ratio	Geometric progression (typically $2\times$–$4\times$), balancing log-space coverage against compute budget	Engineering practice

The extrapolation multiplier is the ratio of target training FLOPs to the largest fitting point's FLOPs. Among public examples, Delphi fits on 3×10^{18}–3×10^{20} FLOPs and validates across $3\times$–$333\times$ holdouts; Llama 3, 2024 §3.2.1 fits on 6×10^{18}–10^{22} FLOPs (40M–16B) and extrapolates ~3,800× to 3.8×10^{25} FLOPs. As the extrapolation multiplier grows, second-order functional-form misspecification and late-training numerical instabilities dominate total error; when target compute sits orders of magnitude beyond the largest holdout, insert an intermediate-scale dress rehearsal before launching the full run (§10.6).

5.3 Training Volume Tiers

Token horizon tiers should bracket the target model's intended TPP. The classic reference is Chinchilla's IsoFLOP method (Hoffmann et al., 2022): across several fixed compute budgets C, train models of varying size N, fit a smooth curve to loss versus $\log N$, and read off the minimum (N_{opt}, D_{opt}) at each compute level.

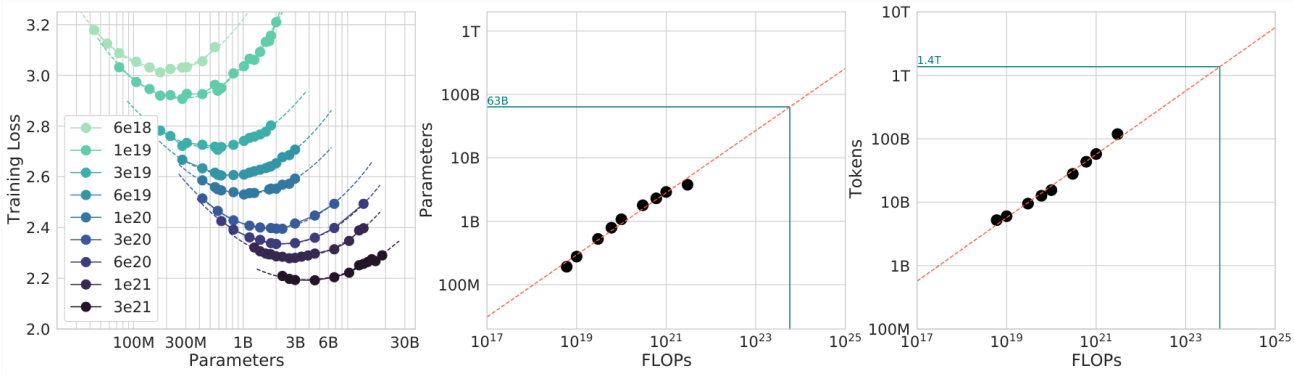


Figure 3. IsoFLOP results of Chinchilla. Left: an approximate quadratic fit of loss against log N at each compute budget, with the minimum corresponding to the optimal size; middle and right: power-law fits of the optimal parameter count and token count against compute. Source: Hoffmann et al., 2022, Fig. 3.

Across 400+ models (70M–16B parameters, 5B–500B tokens), Chinchilla established:

$$N_opt \propto C^a, \qquad D_opt \propto C^b, \qquad a \approx b \approx 0.5$$

Along the compute-optimal frontier, model size and training tokens scale in roughly equal proportion. Note that Chinchilla's $D/N \approx 20$ rule-of-thumb counts total parameters including embeddings; expressed in terms of trunk parameters N_{body}, the optimal TPP is higher and shifts with corpus quality and recipe. Using IsoFLOP curves at 3.8×10^{25} FLOPs, the Llama 3 team estimated an optimal size of ~402B and selected 405B (Llama 3 §3.2.1).

Adapt the token horizon tiers to the deployment and data regime:

- Near compute-optimal: 0.5×–4× Chinchilla brackets both sides of the 1× optimum; Fantastic Optimizers uses 1×, 2×, 4×, and 8× Chinchilla tiers to cover mild over-training.
- Heavy over-training: Gadre et al., 2024 validate across 104 models that power-law loss scaling holds reliably up to 32× Chinchilla. Pushing further to 10,000 TPP across 47 models, Sardana et al., 2024 find that while loss continues to improve, Chinchilla coefficients fitted solely on low-TPP runs overestimate the marginal return of extra tokens at extreme TPP. Projects targeting high TPP must therefore include high-TPP fitting and holdout points.
- Inference-amortized sizing: folding lifetime inference compute into the total cost objective systematically shifts the optimum toward smaller models trained for far more tokens (Sardana et al., 2024). Thus Llama 3 405B is near compute-optimal for training, whereas the 8B and 70B models are heavily over-trained beyond 15T tokens to minimize serving latency and memory footprint (Llama 3 §1; see §10.5).
- Data-constrained regime: when target tokens D exceed available unique tokens U, annotate epoch counts along the horizon axis and include a repetition decay term in the fitted law (§7.3).

5.4 Intermediate Checkpoints, Random Seeds, and Shared Trajectories

Including intermediate checkpoints from a single run (especially under WSD schedules) multiplies the number of data points at zero extra training cost, but introduces two statistical pitfalls:

First is contamination from early transient dynamics. Early in training, loss drops steeply and deviates from the asymptotic power law; Choshen et al., 2025 show across hundreds of models that discarding the first 10% of checkpoints or the first 10B tokens substantially reduces extrapolation error. Lock this truncation rule before fitting, or split model sizes into fit, validation, and test tiers and tune the cutoff on the validation tier (Lourie et al., 2026)—never tune truncation cutoffs on the final holdout.

Second is serial autocorrelation and shared prefixes. Checkpoints along a single trajectory are strongly autocorrelated and carry far fewer effective degrees of freedom than independent runs (§8.3). Likewise, when multiple WSD decay branches fork off a shared stable trunk, they inherit identical prefix noise: do not double-count trunk FLOPs in budget tables, and treat shared-trunk branches as a correlated cluster during bootstrap resampling.

5.5 Budget Allocation and Follow-Up Experiments

Total Ladder compute must cover pilot runs, hyperparameter grids, multi-seed replications, annealing branches, and failed-run retries. A disciplined budget allocation proceeds in stages:

1. Run small pilot trials to measure random seed variance, hardware throughput, and rough effect sizes across candidates;
2. Partition the main budget across hyperparameter tuning, grid fitting, and holdout validation while holding back 15%–20% as a reserve;
3. Deploy reserve compute based on mid-course diagnostics: add random seeds at small scales if seed noise dominates; add grid points in the divergence zone if competing functional forms (e.g., Chinchilla vs. Skaling) split at high TPP or large N; or extend training horizons if candidate loss curves cross late in training.

Whether marginal budget is better spent adding a larger model size or running extra random seeds at small scale depends on the ratio of seed noise to extrapolation span (Choshen et al., 2025).

5.6 MoE Experimental Axes

MoE architectures introduce too many degrees of freedom to sweep in a full Cartesian product across active parameters, total parameters, and expert granularity. Instead, factor the MoE Ladder into three orthogonal axes:

1. Scale Ladder: lock sparsity S and expert granularity, scaling N_{active} and D together;
2. Sparsity Ladder: lock N_{active}, D, and active expert count E_{active}, sweeping total experts E_{total} alone;
3. Granularity Ladder: lock N_{active} and N_{total}, inversely varying individual expert size and active expert count (Krajewski et al., 2024).

For example, Kimi K2, 2025 swept E_{total} at fixed N_{active} and FLOPs to fit a sparsity scaling law, showing that raising sparsity from 8 to 48 steadily reduces the compute needed to hit a target loss—leaving cross-node communication and inference memory bandwidth as the binding constraints.

Keep data from the three axes separate unless the fitted equation explicitly models those variables, and include iso-N_{active} or iso-FLOP dense baselines alongside the Scale Ladder to track how MoE's effective compute multiplier evolves with scale.

6. Hyperparameter Search and Training Configuration

Except when evaluating a fixed-rule recipe (Objective 1 in §2.1), compute allocation, candidate comparison, and hyperparameter extrapolation all require calibrating how optimal hyperparameters $(\eta_{opt}, B_{opt}, \lambda_{opt})$ shift with (N, D).

6.1 Hyperparameter Search Objective and Near-Optimal Region

Why spend heavy compute grid-searching small models while only checking a few local points at large scale? Lourie et al., 2026 and Step Law (Li et al., 2025) establish a fundamental asymmetry: small models are sharply sensitive to suboptimal hyperparameters, whereas large models exhibit a wide, flat optimal basin. On small models, off-optimal hyperparameters noticeably inflate loss and warp the fitted power-law exponent—clean scaling laws emerge only along the Fully-Tuned Frontier. At large scale, by contrast, Qwen3.8-Next, 2026 multiplied and divided LR by $\sqrt{2}$ and raised BSZ by 25% on a 156B-A7B model, observing final training loss variations of at most 7×10^{-4}.

This asymmetry dictates search strategy: concentrate dense grid searches on cheap small models so that every fitting point's optimum sits strictly in the interior of the search grid (with local perturbation deltas smaller than seed noise), then rely on narrow local verification around the extrapolated values at large scale (§6.7).

6.2 Parameterization and Optimizer Transfer

- Width transfer: μ-Transfer (Yang et al., 2022) rescales per-layer initialization and learning rates so that optimal LR transfers zero-shot across model width; modifications for nonzero weight decay are detailed in Appendix G.1.2 of the paper and in Power Lines. Because μP alone does not handle scaling across batch size or token horizon D, combine μP parameterization with empirical (N, D) power laws for BSZ and horizon.
- Depth transfer: Bordelon et al., 2023 scale residual branches by $1/\sqrt{\text{depth}}$ under μP to achieve width-and-depth hyperparameter transfer in ResNet and ViT on CIFAR-10 and ImageNet. However, Tensor Programs VI (Yang et al., 2023) proves that while Depth-μP holds for blocks with a single layer, fundamental theoretical limitations prevent exact infinite-depth transfer when residual blocks contain multiple non-linear layers (as in standard Transformers). When depth changes in LLMs, recalibrate hyperparameters empirically.
- Cross-optimizer transfer: switching optimizers reshapes the hyperparameter landscape. When scaling Muon to LLMs, Liu et al., 2025 added weight decay and scaled the orthogonalized update RMS to match AdamW's typical 0.2–0.4 range (setting 0.2), allowing tuned AdamW LR and WD values to transfer directly. However, when Qwen3.8-Next, 2026 changed architecture and switched to Muon simultaneously, both optimal LR and optimal BSZ shifted, underscoring the need to re-verify hyperparameter laws at small scale after optimizer changes.

6.3 Scaling Law for LR and BSZ

When a Ladder runs at a single fixed D/N ratio, optimal learning rate and batch size can be fitted as simple univariate power laws of compute C:

$$\eta_opt = a \cdot C^{-\alpha}, \quad B_opt = b \cdot C^{\beta}$$

as in DeepSeek LLM, 2024. Once a Ladder spans multiple D/N ratios, however, C conflates model size and training horizon, so N and D must be decoupled:

Approach	η_{opt} Formula	B_{opt} Formula	Key Findings & Scope
Step Law (Li et al., 2025)	$c \cdot N^{-\alpha} D^{\beta}$	$0.58 D^{0.571}$	Validated across 3,700+ models and diverse D/N; regression tests confirm B_{opt} depends on D and is nearly invariant to N (Appendix A.5)
Power Lines (Bergsma et al., 2025)	Coupled via timescale τ (§6.4)	$\propto D^{0.4}$ (weakly dependent on N)	Unifies (η, B, λ) via EMA timescale τ, subject to maximum stable LR and critical batch limits
Token Horizons (Bjorck et al., 2024)	$c \cdot N^{-\alpha} D^{-\beta}$	Fixed BSZ	Holding N and BSZ fixed, longer token horizons D require a lower peak LR

Notice the striking sign flip on the exponent of D in η_{opt}: Token Horizons finds $D^{-\beta}$ while Step Law finds $D^{+\beta}$. The discrepancy stems from their experimental controls: Token Horizons holds batch size constant as D increases (so more steps require a smaller step size to settle), whereas Step Law co-scales $B_{opt} \propto D^{0.571}$ as D grows—and the variance reduction from a larger batch size outweighs the longer horizon, pushing optimal peak LR slightly upward with D.

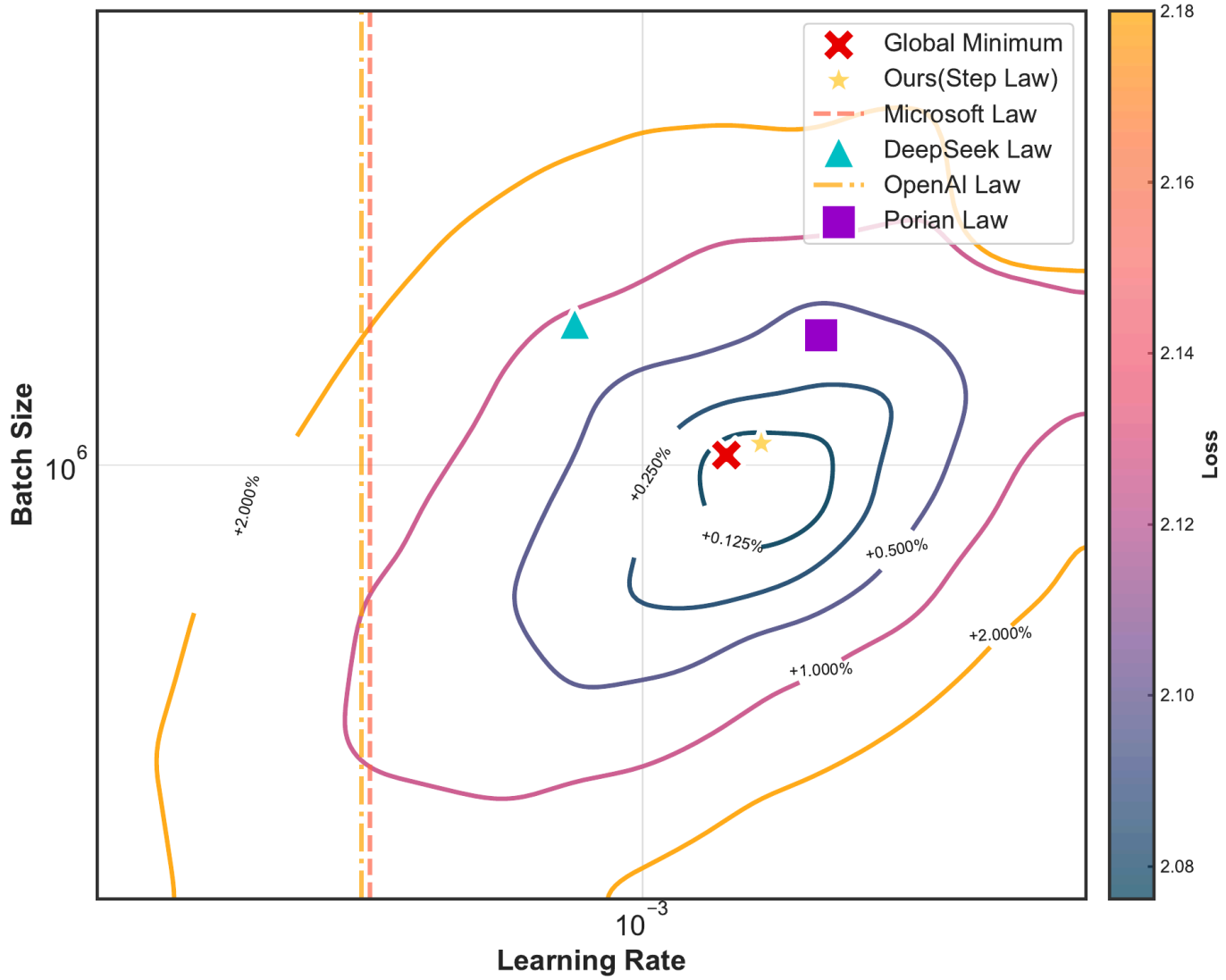


Figure 4. Step Law under the test condition N = 1B, D = 100B, comparing the configuration predicted by the hyperparameter formula with the experimentally determined optimal configuration. The contours are obtained from 120 training experiments with different LR and BSZ combinations; this condition lies outside the fitting range of the paper. The comparison shown is limited to the training recipe and learning rate schedule used in the paper. Source: Predictable Scale: Part I, 2025, v3, Fig. 1 (PDF page 1).

On optimal batch size, Step Law and Power Lines converge on a crucial insight: B_{opt} grows as a power law of training tokens D (exponent ~0.4–0.57) and is largely independent of model size N.

Alongside B_{opt}, Power Lines characterizes the critical batch size $B_{crit} \propto D_{min}^{0.5}$, where D_{min} is the minimum token count needed to reach a target loss at small batch size. As shown in Figure 5, the training tokens D and optimization steps S required to hit a given loss trace a hyperbola: $B = B_{crit}$ marks the knee of the curve where training consumes $2D_{min}$ tokens, beyond which further increases in batch size yield diminishing step reductions at steep token and FLOP cost.

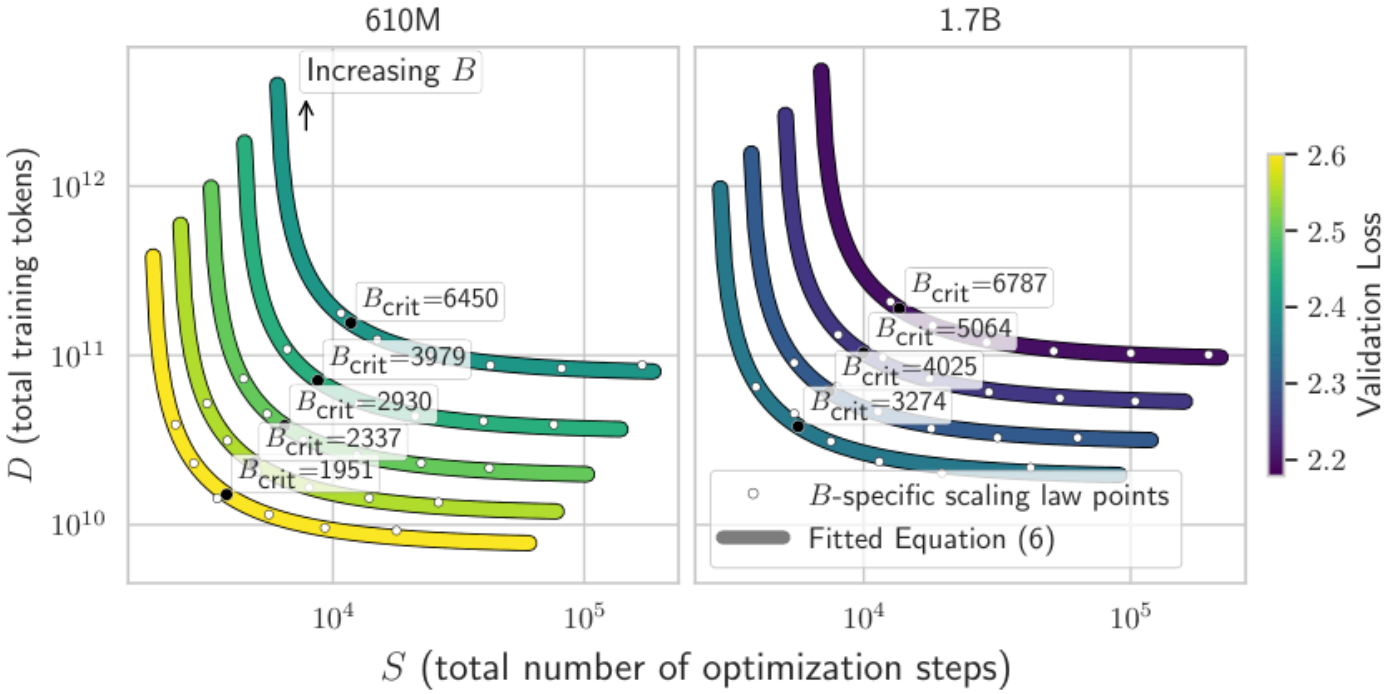


Figure 5. Hyperbolic relationship between training tokens and steps required to reach the same target loss, for 610M and 1.7B models on the left and right respectively, with color indicating loss. B_crit marks the trade-off turning point between token efficiency and step count. Source: Bergsma et al., 2025, v2, Fig. 4.

Fortunately, the near-optimal batch size basin is forgiving: Schaipp, 2026 shows that batch sizes within a ~5% compute overhead span a $4\times$ factor—corresponding to $[B_{opt}/2, 2B_{opt}]$ under a log-symmetric model—giving ample leeway to align batch sizes with hardware parallelism multiples.

Two practical batch-size rules apply across all runs:

- Express batch size in total tokens rather than sequence count when changing sequence length, while accounting for shifts in attention FLOPs and document packing.
- Many flagship runs employ batch size warmup/ramp-up during training (e.g., Llama 3 §3.4.1 ramps BSZ in stages from 4M to 16M tokens, and Nemotron-4, 2024 uses a similar ramp). If Ladder runs use a constant batch size while the target run ramps BSZ, test how the ramp alters early loss trajectories and effective hyperparameter transfer.

6.4 Weight Decay

Several prominent recipes hold weight decay constant across scales (e.g., Kimi K2, 2025 fixes $\lambda = 0.1$; OLMo 3, 2025 uses AdamW with fixed WD and zero decay on embeddings; Qwen3, 2025 scales LR and BSZ without scaling WD).

Across wide TPP ranges, however, optimal WD is not constant. Han et al., 2026 show that the weight decay minimizing pretraining loss decreases monotonically as TPP grows, whereas maintaining a somewhat higher WD preserves effective weight rank and downstream plasticity during post-training.

Under AdamW, (η, λ, B) can be calibrated jointly through the EMA timescale $\tau = B/(\eta\lambda D)$: Power Lines demonstrates that optimal configurations maintain a constant τ_{opt}, so when varying B or D at fixed μP learning rate η, scaling $\lambda \propto B/D$ preserves the weight memory horizon—provided η remains below the maximum stable learning rate (Power Lines §2.4).

6.5 LR Schedule

Pretraining runs typically rely on one of two schedules:

- Cosine schedule: smooth cosine decay after a brief warmup down to 0 or ~10% of peak LR, well suited when the target token budget D is fixed upfront.
- WSD (Warmup-Stable-Decay): holds LR constant across most of training before decaying sharply at the end. River Valley (Wen et al., 2024) explains WSD through loss-landscape geometry: the high-LR stable phase advances rapidly along the flat valley floor, while the decay phase settles into the sharp transverse walls. For Scaling Ladders, WSD's chief advantage is trajectory reuse—multiple decay branches can fork off a single stable trunk to sweep several D tiers at a fraction of the cost of independent cosine runs.

Setting the decay phase to 10%–20% of total training steps provides a strong baseline (Tissue et al., 2024); Wang et al., 2025 analyze how annealing shapes and fractions transfer across scales.

6.6 MoE Training Hyperparameters

Because sparse routing changes gradient noise statistics, (η_{opt}, B_{opt}) scaling laws for MoE should be fitted directly on the MoE family rather than copied unverified from an iso-active dense Ladder. Lock router-specific knobs—load-balancing coefficients or bias update rates, FP32 router logits, and capacity factors—according to §4.4, and verify at intermediate scales that expert load distributions and token drop rates remain healthy.

6.7 Search Procedure and Stopping Rules

Hyperparameters exhibit a clear sensitivity hierarchy: peak LR and BSZ dominate first-order loss, LR schedule and WD come second, and numerical stability terms (β_2, ϵ) can be fixed after baseline and large-scale sanity checks. A coarse-to-fine coordinate search keeps compute tractable (model sizes illustrative):

1. Run a full two-dimensional grid over (η, B) at the smallest size (~130M), ensuring the loss minimum lies strictly inside the grid boundaries;
2. Run a narrower grid at a medium size (~500M) to anchor the power-law slope;
3. Verify locally around the extrapolated point at large scale: test LR within $[\eta_{pred}/\sqrt{2}, \sqrt{2}\eta_{pred}]$ and BSZ within $[B_{opt}/2, 2B_{opt}]$, expanding the grid only if a boundary point wins (§6.1, §6.3).

Using the N-invariance of $B_{opt}(D)$ from §6.3 and calibrating λ via the timescale τ from §6.4 collapses the three-dimensional search space into manageable lower-dimensional slices.

Selecting the lowest loss across multiple noisy trials inevitably introduces winner's curse selection bias, overstating the winning configuration's true gain (Cawley & Talbot, 2010). To eliminate selection bias, re-run the winning configuration with a fresh random seed and use that independent evaluation for curve fitting; equally important, never prune candidate trials early based on pre-annealing loss, as rankings routinely flip during LR decay (§6.8).

6.8 Recipe Comparison

Comparing optimizers, architectures, or LR schedules on a Ladder fails most often due to unequal tuning or premature evaluation:

- Match tuning rigor across candidates: Fantastic Optimizers (Wen et al., 2025) shows that many reported optimizer breakthroughs stem from comparing a freshly tuned new optimizer against an untuned default AdamW baseline. Likewise, Kimi K3, 2026 §3.2 found that cosine and WSD schedules peak at very different (η, B) values for the same (N, D); once separate hyperparameter scaling laws were tuned for each schedule, cosine consistently achieved lower final loss than WSD and was chosen as the production default.
- Compare only at the annealed endpoint: Fantastic Optimizers documents frequent rank inversions during LR decay, making mid-stable-phase checkpoints unreliable for candidate elimination.
- Measure how gains decay with scale: in the 8× Chinchilla regime of Fantastic Optimizers, the token-efficiency multiplier of matrix optimizers over AdamW shrank from $1.4\times$ at 0.1B down to $1.1\times$ at 1.2B (and token savings do not

automatically equal wall-clock speedups). Any candidate win on a single small model must be tracked across at least three sizes to verify its advantage does not vanish at scale.

6.9 Training Wrap-Up

Production pretraining rarely ends with a uniform-distribution run. If the delivery pipeline includes late-stage wrap-up operations, incorporate them into the Ladder (or use a validated proxy) so fitted losses match the delivered model:

- Continued training / cooldown mixture shift: switching to a high-quality, reasoning-dense mixture during steep LR decay at the end of training (Nemotron-4, 2024; OLMo 2, 2025; see §11.1).
- Weight averaging: merging independently fine-tuned branches from a shared checkpoint (Model Soups) or averaging checkpoints along a sliding window on a single trajectory (LAWA; Sanyal et al., 2024). Notably, Model Merging (ByteDance Seed, 2025) shows at 1.3B and 13B that checkpoint merging (PMA) during the WSD stable phase closely matches the downstream performance of a fully annealed model, offering a cheap proxy for annealed performance.

7. Data Ladder

Holding model architecture and optimizer fixed, a Data Ladder answers three questions: whether a new corpus is worth adding (§7.1), how to weight domains (§7.2), and how many epochs scarce high-quality data can be repeated before overfitting (§7.3).

7.1 Data Quality and Data Source Evaluation

Comparing pretraining corpora of differing quality, DeepSeek LLM, 2024 established two core findings:

- Higher-quality data shifts compute-optimal allocation toward larger model size N: denser, cleaner corpora reward greater model capacity per token;
- Scaling law exponents are corpus-dependent: coefficients fitted on one dataset version cannot be reused blindly on another.

Consequently, when Kimi K3, 2026 §3.2 updated its architecture, corpus, and training recipe simultaneously, the team rebuilt its scaling laws from scratch to re-tune BSZ, LR, optimal TPP, and aspect ratio.

Screening an individual new data source, however, does not require re-running a full two-dimensional Ladder:

- Checkpoint micro-annealing: OLMo 2, 2025 forks a mid-to-late checkpoint, mixes the candidate dataset with baseline data over a short decay window, and measures downstream lift (noting that micro-annealing captures late-stage marginal value rather than full-run rankings).
- Small-model domain ablations: Kimi K3 §3.1 sets per-domain sampling weights via controlled ablations on small models.
- Targeted synthetic domain injection: adding the legal synthetic corpus released with Nemotron 3 Ultra, 2026 into Nemotron 3 Nano pretraining lifted average LegalBench proxy accuracy from 64.6 to 74.7.

7.2 Data Mixture Ladder

A Data Mixture Ladder fixes N, D, and optimizer settings while varying the domain weight vector $\mathbf{w} = (w_1, \dots, w_m)$. Synthesizing Olmix, 2026, OLMo 3, 2025 §3.4.4, and the Marin data pipeline, 2026, an end-to-end mixture optimization pipeline proceeds in seven steps:

1. Bucket partitioning and inventory: partition corpora by source, domain, or fine-grained instance tags (such as educational value, code, language, and safety; Qwen3, 2025), re-tallying deduplicated unique tokens N_j whenever quality filters change (Marin data pipeline, 2026). If the production run shifts mixtures across stages (e.g., Nemotron 3

Super, 2026 emphasizes diversity for the first 80% of its 25T tokens and high-quality sources for the final 20%), optimize each stage's mixture separately (§11.1).

2. Baseline anchors: always benchmark against natural token-proportional sampling and repetition-capped UniMax; as Marin notes, poorly constrained learned mixtures frequently underperform both baselines.
3. Small-scale proxy design:
   - Proxy size: at 5× Chinchilla horizon, proxy models of 15M parameters and above achieve 0.89 Spearman rank correlation with a 1B target model, whereas a 1M proxy drops to 0.73 (Olmix); OLMo 3 uses 30M proxies trained for 3B tokens.
   - Proportional pool downscaling: if a tiny proxy samples from the full multi-terabyte pool, it never experiences the multi-epoch repetition that hits the flagship model. Marin therefore shrinks each bucket's available token pool in exact proportion to the proxy's active parameters, forcing the proxy to encounter the exact same repetition rate (791 tokens per active parameter) as the target ladder.
   - Sample count and model-merging alternatives: fitting a log-linear response surface across m domains requires at least $3(m+1)$ proxy mixtures sampled from a Dirichlet distribution centered on the natural prior. To cut proxy training cost, DeMix, 2026 trains one component model per domain and evaluates arbitrary mixtures via weighted model merging, achieving higher rank consistency than retrained small proxies.
4. Per-task response regression: Olmix shows that fitting separate log-linear models per evaluation task and then aggregating achieves 0.983 correlation on held-out mixtures, far beating the 0.866 correlation of fitting a single aggregate score directly. Moreover, the regression form must capture non-monotonic saturation: Marin observes that raising a bucket's weight initially lowers loss, plateaus, and then degrades loss once small bucket capacity forces heavy repetition; because $\sum w_j = 1$, single-bucket marginal correlations are confounded by competing buckets and can flip sign between pretraining and cooldown (Marin).
5. Repetition-constrained solver: solve for the optimal mixture under explicit per-bucket epoch caps $w_j \leq kN_j/R$ (R target tokens, k max epochs; §7.3) plus a light KL regularizer ($\lambda = 0.05$) toward the natural distribution.
6. Cross-scale confirmation: validate the solved mixture against proportional and UniMax baselines on intermediate-scale models before launching the flagship run (Marin).
7. Incremental mixture reuse: when only a subset of buckets is updated, Olmix's mixture reuse freezes the relative weights among unchanged buckets and re-runs proxies only for the modified subspace. Across 5 updates scaling to 64 domains (1B model at 100B tokens), mixture reuse delivered an 11.6% gain over natural sampling—capturing 95% of full-recomputation gains while cutting proxy runs by 74%—and powered three mixture iterations in OLMo 3.

7.3 Data-Constrained Training and Repetition

When target training volume D exceeds available unique tokens U, multi-epoch repetition is unavoidable. The prerequisite for studying repetition is global cross-source deduplication: the Marin data pipeline, 2026 performs global deduplication across all corpora so "1 epoch" is a true controlled variable, uncovering massive hidden overlap between Nemotron-CC and its synthetic rewritten variants.

Assuming the marginal value of repeated tokens decays exponentially with epoch count, Muennighoff et al., 2023 replace raw tokens D with effective data volume D':

$$D' = U_D + U_D \cdot R_D^* \left(1 - e^{-R_D/R_D^*}\right)$$

where U_D is unique tokens, R_D is the number of additional repetitions beyond the first epoch ($R_D = 0$ for a single epoch), and R_D^* is the fitted decay half-life scale (empirically ~4). As $R_D \to \infty$, effective volume saturates at $U_D(1 + R_D^*)$.

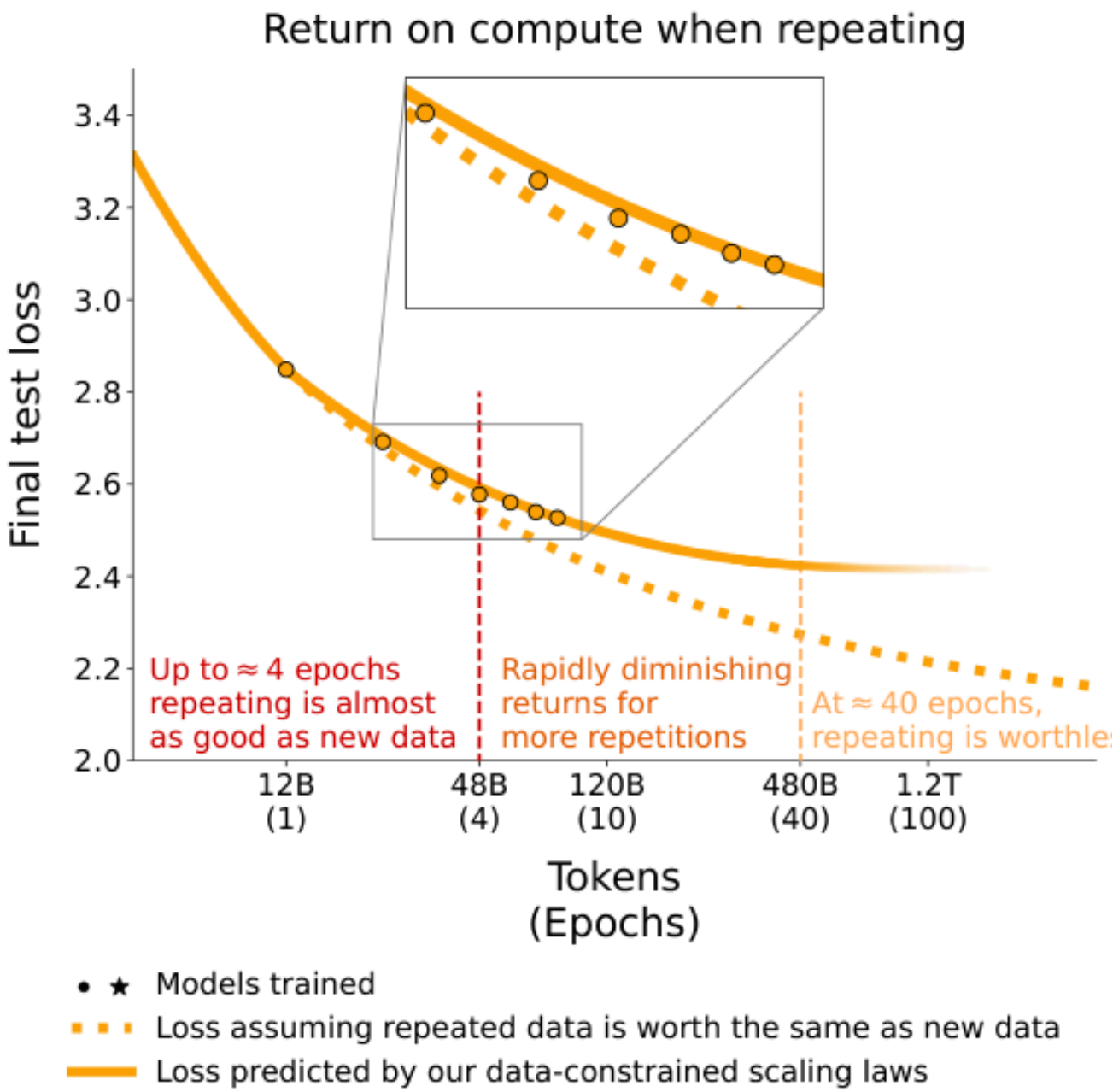


Figure 6. Marginal returns of data repetition (4.2B model, 12B unique tokens). The first few epochs of repetition yield returns close to those of new data, after which marginal returns decline and saturate after roughly 40 epochs. Source: Muennighoff et al., 2023, v5, Fig. 1 left.

Recent empirical and theoretical work clarifies when repetition helps and when it hurts:

- Overfitting scales with parameter count N and inverse pool size $1/U$: Yan et al., 2025 show analytically in linear regression that optimal repetition grows logarithmically with dataset size, while Lovelace et al., 2026 and Xue et al., 2023 demonstrate that larger N, smaller U, and higher repetition jointly accelerate memorization. Consequently, when unique tokens U are fixed, scaling N too aggressively worsens repetition overfitting, shifting the compute-optimal allocation toward smaller N.
- Quality alone does not prevent multi-epoch overfitting, but dropout helps: Xue et al., 2023 find that at fixed dataset size, higher data quality does not immunize a large model against multi-epoch overfitting, whereas re-tuning dropout (v2 Table 4) substantially mitigates repetition degradation.
- Scarce domains tolerate far more repetition when diluted in a large general mixture: across 2,000+ runs, Sedova et al., 2026 show that a small target-language or specialist corpus mixed into a massive general dataset can be repeated 15–

20 times with positive returns, and their repetition-aware mixture scaling law extrapolates reliably from small to large scale.

- Enforce quality-tiered epoch caps: production recipes cap maximum epochs per bucket (§7.2). Kimi K2.5, 2026 caps per-source epochs when continuing joint pretraining from a late Kimi K2 checkpoint; OLMo 3 (§3.4.4, Appendix A.2.5) upsamples only high-quality buckets up to 7 epochs, outperforming aggressive threshold filtering.
- Synthetic rewriting stretches unique token horizons: paraphrasing high-value knowledge corpora into diverse surface forms delays memorization saturation. On SimpleQA at an early checkpoint, Kimi K2, 2025 §2.2 scored 23.76 when repeating raw text for 10 epochs, 27.39 when rewriting once and training for 10 epochs, and 28.94 when generating 10 distinct rewrites trained for 1 epoch each (capping production rewrites at 2 passes per corpus, a practice retained in Kimi K3 §3.1). Track rewritten tokens separately from raw unique tokens U.

8. Loss Scaling Law Fitting

Before fitting $L(N, D)$, verify that the three confounding factors identified by Porian et al., 2024 have been resolved:

1. Compute C includes output-head FLOPs and is logged separately from the trunk parameter count N_{body} (§3.1);
2. Warmup steps scale proportionally with training volume rather than remaining fixed (§4.1);
3. For Objectives 2–4, every fitting point has been tuned onto the Fully-Tuned Frontier (§6.1).

8.1 Functional Form

Chinchilla's classic additive law $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ imposes zero interaction between model size and token horizon ($\partial^2 L/\partial N \partial D \equiv 0$). Analyzing dense (N, D) grids, Skaling (Videau et al., 2026) finds a consistently negative mixed partial derivative—meaning N and D complement each other—and introduces a single outer coupling exponent k:

$$L(N, D) = \left(\frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}\right)^{k} + E$$

Setting $k = 1$ recovers Chinchilla, whereas empirical fits yield $k \approx 0.31$–0.45. Crucially, adding just 1 extra free parameter k leaves the closed-form algebraic expression for compute-optimal (N_{opt}, D_{opt}) unchanged.

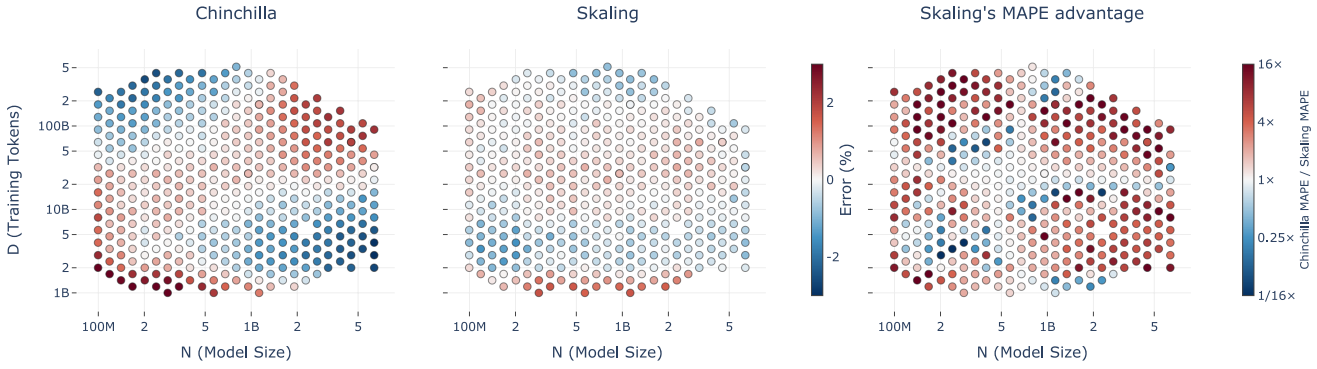
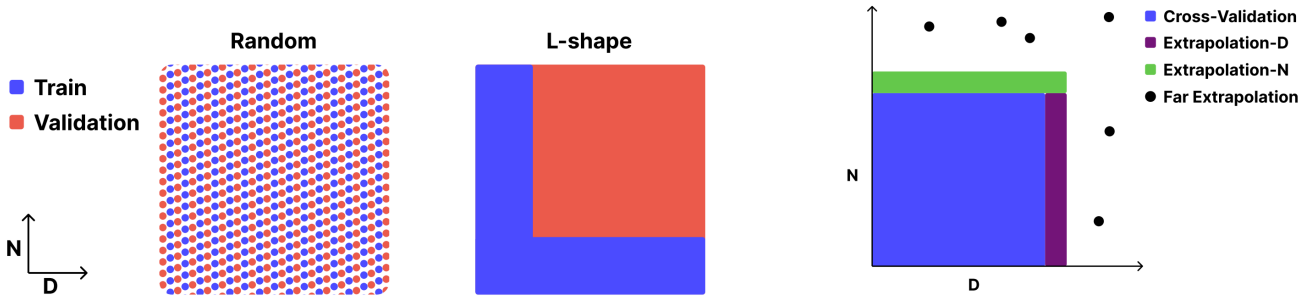


Figure 7. Prediction residuals of the two forms on the same (N, D) grid, with each point a single training run. Left and middle: signed percentage error, sharing a color scale; the Chinchilla residuals are saddle-shaped and grow toward the four corners, while Skaling is near zero across the whole grid. Right: the ratio of the two errors; Skaling is more accurate on 76% of configurations, with a median ratio of 2.2×. Source: Videau et al., 2026, Fig. 1.

As Figure 7 illustrates, Chinchilla's additive form produces a pronounced saddle-shaped residual pattern that blows up toward the four corners of the (N, D) grid (extreme under-training and over-training), whereas Skaling flattens residuals near zero across the entire surface. Compared against the nine-parameter Farseer (Li et al., 2025) form (which makes data-side coefficients and exponents explicit functions of N), the six-parameter Skaling law achieves lower interpolation and single-axis extrapolation error on both the Farseer and SK-Grid benchmarks (Table 1). Fit both Skaling and Chinchilla as default candidates and select between them on a held-out validation slice.

When all runs share a single fixed TPP ratio $M = D/N$, the surface collapses to a one-dimensional power law $L = G(M)/C^{\gamma} + E$; any cross-TPP extrapolation or compute-allocation solve requires the full bivariate (N, D) surface.

Better functional forms also unlock much cheaper experimental grids. In Skaling's ablations, an L-shape sparse layout—sweeping N only at small D and sweeping D only at small N (Figure 8)—costs just $1/5$–$1/10$ the compute of a full grid; Chinchilla's additive form suffers severe extrapolation drift into the large-(N, D) interior from an L-shape, whereas Skaling extrapolates accurately.



(a) Sampling strategies. **(b)** Evaluation regimes.

Figure 8. Sampling strategies and evaluation regions of Skaling. Left: Random holds out validation points at random; L-shape covers different N at small D and different D at small N. Right: regions for interpolation, extrapolation along N or D, and joint extrapolation. The horizontal axis is D and the vertical axis is N. Source: Videau et al., 2026, v1, Fig. 4 (PDF page 5).

8.2 Loss Curves and Annealing Scaling Law

To predict the entire step-by-step training curve rather than just the final endpoint, Tissue et al., 2024 parameterize loss at step s directly in terms of the learning rate schedule:

$$\hat{L}(s) = L_0 + A \cdot S_1(s)^{-\alpha} - C \cdot S_2(s)$$

where $S_1(s) = \sum_{i \le s} \eta_i$ is cumulative learning rate area (capturing forward optimization progress) and $S_2(s)$ is cumulative LR decay convolved with an exponential forgetting kernel (capturing annealing settling gains). Taking the LR schedule as an explicit functional input allows a single fit to leverage dozens of intermediate evaluation points along a trajectory and predict full loss curves across cosine schedules, WSD branches, and continued-training re-warmups (§4.7 of the paper).

8.3 Fitting Protocol

Running a single pass of `scipy.optimize.curve_fit` on raw loss values is notoriously fragile: both the Chinchilla replication by Besiroglu et al., 2024 and the survey in (Mis)Fitting (Li et al., 2025) demonstrate that non-linear power-law fits are highly sensitive to numerical specification, where minor changes in initialization, loss weighting, or outlier handling swing extrapolated (N_{opt}, D_{opt}) ratios by severalfold. Standardize the fitting pipeline around three practices:

- Target transformation and robust loss: specify upfront whether residuals are minimized on L or $\log L$ and whether irreducible entropy E is fitted freely or bounded by a prior; use Huber loss to limit the leverage of noisy points and weight trajectories inversely by checkpoint count so long runs do not dominate short runs.
- Multi-start global optimization: because power-law coefficients and exponents trade off along curved valleys, local gradient solvers easily trap in poor local minima; optimize in log-parameter space with explicit box bounds across hundreds of random initializations (multi-start L-BFGS-B).
- Block bootstrap and decision propagation: when estimating confidence intervals, resample at the level of independent runs (or shared-trunk branch clusters) rather than treating serially correlated checkpoints as independent samples. For every bootstrap replicate, re-solve the target (N_{opt}, D_{opt}) or candidate margin (§2.3) so uncertainty is reported directly on the engineering decision rather than just on raw curve parameters.

8.4 Fitting Diagnostics and Failure Handling

Before trusting an extrapolation, run three diagnostic checks on the fitted surface:

- Inspect two-dimensional residual structure: plot signed residuals $\hat{L} - L$ against $\log N, \log D$, and TPP. A saddle pattern (as in Figure 7 left) or monotonic tail curvature signals functional-form misspecification that will compound under extrapolation.
- Guard against inflated degrees of freedom: verify that autocorrelated intermediate checkpoints are down-weighted or block-bootstrapped (for instance, Delphi bootstraps strictly over the fitted optima of each IsoFLOP parabola, eliminating intra-trajectory correlation).
- Decompose holdout error: compare holdout residuals against the multi-seed standard deviation at fixed configuration —if holdout error approaches seed noise, the fit has hit the noise floor; if systematic bias dwarfs seed noise, revisit the early-checkpoint truncation cutoff (§5.4) or add grid points along the failing axis.

8.5 MoE Fitting

Fit MoE Ladders by aligning the regression variables with the orthogonal axes of §5.6:

- On a Scale Ladder with fixed sparsity S and expert granularity, simply substitute per-token active parameters N_{active} for N in the bivariate scaling law (logging N_{total} alongside).
- When sweeping expert counts or granularity, include sparsity S and granularity as explicit variables in the joint functional form (Krajewski et al., 2024; Ludziejewski et al., 2025)—never pool runs of differing sparsity into a single (N_{active}, D) fit.
- Compare the fitted MoE surface against an independently fitted dense baseline in equivalent-compute coordinates to track how MoE's compute multiplier scales with model size.

9. Downstream Task Prediction

9.1 Method Routes

Extrapolating downstream benchmark accuracy from small pretraining runs generally follows one of three routes:

Prediction Route	Modeling Chain	Representative Works	Strengths & Limitations
Loss → Accuracy	Extrapolate general pretraining loss from (N, D), then map loss to benchmark score via a monotonic curve	Gadre et al., 2024 (power-law error rate vs. perplexity); Delphi (sigmoid mapping)	Simplest pipeline; vulnerable to distribution shift between general pretraining loss and specialized tasks

Prediction Route	Modeling Chain	Representative Works	Strengths & Limitations
$(N, D) \rightarrow$ Task NLL $\rightarrow$ Acc	Two-stage fit: extrapolate the negative log-likelihood (NLL) of ground-truth task answers from compute, then map NLL to discrete accuracy	Bhagia et al., 2024 (OLMo Task Ladder); Llama 3 §3.2.1 (extrapolated to 405B)	Eliminates domain mismatch between pretraining corpus and target task; stage-two NLL-to-accuracy noise varies widely across benchmarks
Difficulty-Bucketed End-to-End	Cluster benchmark items by small-model pass rates, fit compute-to-accuracy curves on non-degenerate clusters, and map back to the full test set	COD (Xu et al., 2026, v4) (difficulty feature clustering); GPT-4 Technical Report (HumanEval difficulty bucketing)	Solves the zero-signal problem on hard reasoning benchmarks where small models score near zero; requires multi-sample pass rates and stable item clustering

In Llama 3 §3.2.1, stage one fits a linear relationship between normalized correct-answer NLL and $\log \mathrm{FLOPs}$ using only Ladder runs up to 10^{22} FLOPs; stage two pools the Ladder models with existing Llama 2 checkpoints to fit a sigmoid curve from NLL to accuracy, yielding tight predictions for 405B on benchmarks like ARC-Challenge (slightly conservative relative to actual). In practice, Delphi's IsoFLOP-plus-sigmoid mapping provides a fast baseline across standard suites, while two-stage NLL or difficulty bucketing can be layered onto flagship reasoning tasks.

9.2 COD Framework

Why do aggregate accuracy curves on hard benchmarks like MATH look flat at small scale and then kink sharply upward? Because a benchmark mixes easy items that saturate early with hard items where small models score strictly 0. COD (Xu et al., 2026, v4) decomposes an unpredictable full benchmark into predictable difficulty subsets in four stages:

1. Multi-sample difficulty clustering: sample each problem multiple times across a suite of small models, use the vector of per-model pass rates as the problem's difficulty signature, and cluster items into difficulty tiers;
2. Per-cluster curve fitting: fit a double-exponential curve $E[\mathrm{Acc}(C)] = g + (1-g) \cdot e^{-aC^{-b}-c}$ (with random-guessing floor g) to each cluster;
3. Subset extrapolation: discard degenerate clusters that remain stuck at the guessing floor or already saturated at small scale, extrapolate the informative mid-difficulty clusters to target compute C, and take a size-weighted average;
4. Subset-to-full mapping: fit a monotonic calibration curve mapping the extrapolatable subset score back to full-benchmark accuracy.

Across 8 benchmarks, COD v4 predicts 70B model accuracy with a mean absolute error of just 1.55 percentage points (Table 1 of the paper).

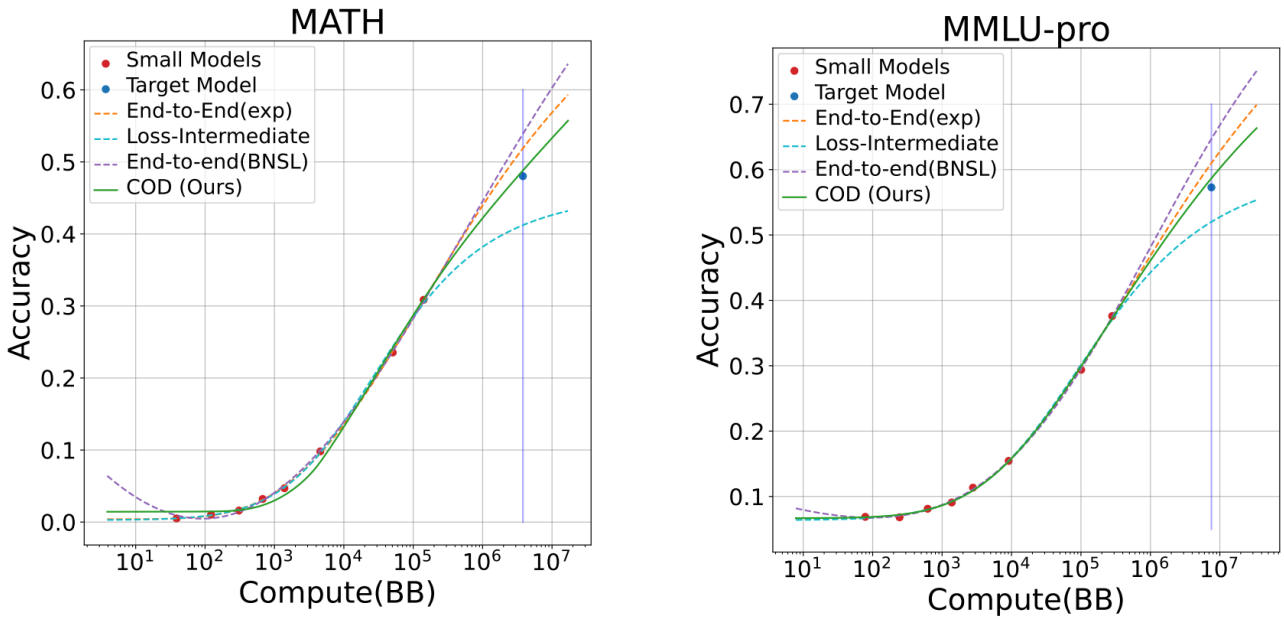


Figure 9. COD v4 predicted curves on MATH and MMLU-pro versus 70B measured values, including COD, Loss-Intermediate, End-to-End(exp), and End-to-end(BNSL). Red dots are small-model results, blue dots are measured values of the target model. Source: Xu et al., 2026, v4, MATH and MMLU-pro subplots of Fig. 4 (PDF page 9).

Keep four boundary conditions in mind when deploying COD:

- Small benchmarks produce noisy clusters, and major changes in architecture or corpus mixture can reshuffle item difficulty rankings, requiring cluster stability checks;
- When target training includes a continued-training mixture shift and annealing phase, small proxy models must match the two-stage data distribution and TPP trajectory (v4 Appendix D–E);
- Cross-architecture transfer in v4 works with wider error bars: using dense small-model clusters to predict a 32B-active MoE target yielded mean and maximum absolute errors of 3.11 and 8.11 percentage points, respectively (§5.3.1, Table 2);
- While open-ended CoT benchmarks show strong empirical fits, theoretical guarantees for non-unique reasoning paths remain an open problem (Appendix H).

9.3 Limits of Predictability

Even with sophisticated two-stage or clustering methods, some discrete benchmarks resist accurate extrapolation from small models. Schaeffer et al., 2024 pinpoint why: on multiple-choice tasks, discrete accuracy depends not only on the probability assigned to the ground-truth answer, but also on how probability mass concentrates on specific competing distractors. Each step from continuous cross-entropy to argmax thresholding discards information about distractor trajectories, weakening the statistical link to compute.

Downstream task predictions must therefore pass the signal-resolution checks of §2.2 and be validated independently on large-scale holdouts; whenever a benchmark fails holdout validation, anchor engineering decisions on continuous task BPB or pretraining loss rather than noisy point predictions.

10. Extrapolation Validation and Launch Decisions

10.1 Holdout Validation

Holdout runs must sit along the actual direction of extrapolation (larger N, longer D, or higher compute C), ideally spanning multiple progressive extrapolation steps as budget permits (§5.2). By fitting on 3×10^{18}–3×10^{20} FLOPs and stepping holdouts up to 10^{23} FLOPs, Delphi caught both the loss drift at $33\times$ extrapolation and the catastrophic divergence at $333\times$ extrapolation (Figure 1). Without multi-tier out-of-sample holdouts, a clean power-law fit is merely an in-sample interpolation.

Enforce one strict rule: a holdout can only be used once as an unbiased test. Once holdout errors are used to tweak the recipe, adjust truncation cutoffs, or swap functional forms, those runs become development data, and signing off on a flagship launch requires fresh out-of-sample validation (§2.3).

10.2 Combined Validation

When individual ablations on architecture, optimizer, hyperparameter scaling, and data mixing each show positive gains, never assume their improvements add linearly (Marin follow-up; OLMo 3, 2025 Appendix A.2.5). Component changes routinely draw from overlapping gain pools or interact negatively—for instance, a more aggressive optimizer can accelerate memorization on repeated buckets, and a new data mixture can shift optimal LR and weight decay. Before freezing the production recipe, assemble all winning changes into a single combined candidate and validate it end-to-end at the target TPP and repetition regime.

10.3 Stability Stress Testing

The biggest blind spot of a small-scale Ladder is training stability: because small runs have fewer layers, shorter horizons, and smaller activation magnitudes, numerical overflows, loss spikes, and router collapse that erupt mid-way through a flagship run often remain completely dormant at small scale.

To surface stability margins early, Qwen3.8-Next, 2026 introduces an effective high-LR stress test: run medium-scale models at $2\times$ and $4\times$ the predicted optimal learning rate to deliberately amplify optimization pressure and compare the divergence threshold of candidate architectures and optimizers against the baseline.

Short high-LR stress tests still do not catch every multi-trillion-token late-stage instability. For example, Nemotron 3 Ultra, 2026 §2.7 encountered two late-training divergences: the first traced to low-precision gradient underflow/overflow in the output layer and resolved after restoring FP32 output-layer gradients, whereas the second had no single identifiable root cause and was mitigated by triggering LR annealing early. Continuously log per-layer gradient norms, activation maxima, and spike counts on all finalist candidates, plus max expert load imbalance, token drop rates, and router entropy drift on MoE runs.

10.4 Implementation Consistency Validation

A pervasive engineering failure mode is that small Ladder runs and flagship production runs often execute on different physical code paths. A Ladder might run on single-node FSDP with standard kernels, while the flagship model runs on multi-dimensional TP/PP/EP parallelism with custom fused kernels, chunked gradient accumulation, low-precision communication collectives, and sharded optimizer states. Identical config parameters do not guarantee mathematically identical updates.

Before transferring Ladder conclusions to the production stack, run a four-step parity check:

1. Single-step forward/backward/update parity: load an identical checkpoint and feed the exact same global batch into both the research stack and the production stack, verifying that forward logits, loss, per-layer gradient norms, and single-step parameter updates match within floating-point reduction tolerances;
2. Short-trajectory convergence parity: run both stacks with fixed seeds for several hundred to a few thousand steps, confirming their loss trajectories stay within numerical rounding noise without systematic drift;
3. Parallelism and reduction audit: when changing parallelism degrees or gradient accumulation steps, verify effective global batch size, loss/gradient reduction denominators (especially under variable sequence lengths or padding), global

gradient-norm clipping order, and accumulator precision;
4. Stateful checkpoint resumption: simulate a crash and restart, verifying that first/second optimizer moments, LR scheduler step, RNG states, and distributed dataloader cursors restore exactly without silently repeating or skipping data windows.

For instance, DeepSeek-V3, 2024 §3.3 specifies distinct precision rules for operator inputs, Tensor Core accumulation, and master optimizer states under FP8 training, while Llama 3 §3.3.4 details deterministic dataloader and state recovery across frequent cluster restarts.

10.5 Realized Efficiency and Deployment Constraints

Theoretical FLOP savings rarely translate 1:1 into wall-clock training speedups. The Marin follow-up explicitly separates theoretical efficiency from realized efficiency, and Nemotron 3 Ultra underscores that low-precision kernels and hybrid architectures must be judged by measured hardware throughput.

When selecting the final production configuration, fold both training-side and serving-side physical constraints into the scaling optimization:

- Training-side constraints: measured MFU and tokens/sec on the target cluster, memory headroom under multi-dimensional parallelism, and effective daily progress after amortizing evaluation, checkpointing, and fault-recovery overhead; for MoE, benchmark expert-parallel all-to-all communication latency on the actual interconnect topology.
- Serving-side constraints: given the target production workload (prefill and decode length distributions, concurrency, TTFT and TPOT latency SLAs, and device memory limits), measure inference cost under the actual deployment quantization scheme. Sardana et al., 2024 model training, prefill, and decode costs jointly—plugging measured cluster and serving unit costs into that framework yields the true lifecycle-optimal (N, D) allocation.

10.6 Medium-Scale Trial Run

When the flagship run's compute budget sits more than two orders of magnitude above the largest Ladder holdout (e.g., extrapolating from a 10^{21} FLOPs Ladder to a 10^{24}–10^{25} FLOPs production run), insert an intermediate medium-scale trial run (dress rehearsal) before committing full cluster resources:

- Execute on the exact production codebase, parallelism topology, numerical precision scheme, and combined data mixture;
- Freeze the predicted loss trajectory and confidence band for that intermediate scale before launch;
- Gate the flagship launch on the trial run's loss curve tracking inside the predicted band with clean stability diagnostics (§10.3).

11. Specialized Ladders

11.1 Staged Ladder

Modern pretraining pipelines proceed through distinct stages—general pretraining, quality/reasoning mid-training (or annealing), and long-context extension—each with its own corpus mixture, sequence length, and LR schedule, causing scaling law exponents to shift across stages (Qwen3, 2025; OLMo 3, 2025). Build dedicated stage Ladders wherever quantitative configuration decisions are needed:

1. Pretraining Ladder: starts from random initialization to fit baseline (N, D, η, B) scaling laws;
2. Mid-training / Annealing Ladder: forks from size-matched pretraining checkpoints to optimize mid-training token budgets, re-warmup / decay schedules, and specialist mixtures;
3. Long-context Ladder: forks from mid-training checkpoints to sweep context length, RoPE base frequency scaling, and continuation LR (§11.2).

Stage Ladders are conditioned on their starting checkpoint: two checkpoints with identical aggregate validation loss can differ substantially in domain exposure, optimizer momentum state, and recent LR history, causing downstream recipes to rank differently. Explicitly test across representative parent checkpoints whether resetting optimizer states, re-warming up LR, or carrying over repeated data buckets changes the stage outcome, and verify the chained end-to-end pipeline (§10.2).

11.2 Long-Context Ladder

Extending context length shifts both the attention FLOP share (§3.2) and cross-document packing dynamics. A Long-Context Ladder must control the mixture of short and long sequences, position distributions of key information, and document-boundary attention masks while tracking both long-context gains and any regression on short-context benchmarks. As RULER (Hsieh et al., 2024) demonstrates, passing simple needle-in-a-haystack retrieval tests does not imply competence on multi-hop tracing or context aggregation, so the evaluation suite must span diverse long-context reasoning tasks.

12. Execution Process and Deliverables

12.1 Experiment Records

A Scaling Ladder is only as reliable as its experiment table. Every run should automatically log a complete provenance record: code commit, full hyperparameter config, dataset manifest and mixture hash, tokenizer version, evaluation script version, initialization and data-order seeds, parent checkpoint path (and shared prefix steps), and hardware topology and precision settings. Tag runs terminated early by design, diverged runs, hardware crashes, and bug-invalidated runs with distinct status codes—neither recording pre-divergence transient losses as valid endpoints nor silently deleting diverged runs that mark the stability boundary.

12.2 Handling Deviations During Runs

Once flagship training launches, the Ladder's predicted loss curve (§8.2) and reference trajectories serve as a live flight instrument. Isolated single-point excursions outside the prediction band typically reflect validation sampling noise or local batch fluctuations; sustained divergence across a progress window calls for a structured triage: first audit evaluation scripts, token counters, and config files, next check dataloader ordering, distributed reductions, and kernel precision parity (§10.4), and only then diagnose recipe extrapolation failure and intervene.

12.3 Ladder Maintenance

As cluster infrastructure and training recipes evolve, maintain the Ladder as a living engineering asset:

- Golden regression suite: keep a small set of reference configurations and their historical loss curves across sizes; re-run them whenever deep learning frameworks, fused kernels, communication libraries, or cluster hardware change to verify losses remain within seed variance;
- Versioned coefficients: archive fitted scaling exponents alongside the exact dataset version, model family, and fitting script that produced them;
- Incremental recalibration: when architecture details, optimizers, corpus versions, precision schemes, or tokenizers change, test two or three small grid points against the existing power law first, triggering local recalibration or a full Ladder rebuild only when deviations exceed seed noise.

12.4 Checklist

Experimental Design Phase

- [ ] Identify the target regime (§1.2), decision objective (§2.1), and evaluation protocol (§2.2)
- [ ] Verify that evaluation metrics have clean signal resolution at Ladder scales; include fast SFT/RL checks when delivering a post-trained model (§2.2)
- [ ] Lock acceptance thresholds before inspecting holdouts, expressed in equivalent compute multipliers (§2.3)

- [ ] Separate fixed invariants, prescribed scaling rules, and independent variables, scaling warmup steps proportionally to total tokens (§4.1)
- [ ] Choose the number of model sizes, size span, and multi-tier holdout extrapolation multipliers (§5.2)
- [ ] Bracket the target TPP with token horizon tiers; log unique tokens U and repetition counts under data constraints (§5.3, §7.3)
- [ ] Allocate budget across pilot trials, hyperparameter grids, multi-seed replications, and contingency runs (§5.5)
- [ ] MoE: log N_{total} and N_{active} separately, factor experiments into orthogonal scale, sparsity, and granularity axes, and include dense baselines (§3.1, §5.6)
- [ ] Freeze the dataset version (§7.1) and align global deduplication scope with epoch accounting (§7.3)
- [ ] Benchmark learned mixtures against natural sampling and UniMax while downscaling proxy data pools proportionally (§7.2); optimize multi-stage mixtures per stage (§11.1)

Configuration and Measurement Phase

- [ ] Separate trunk parameters N_{body} for curve fitting from full FLOP accounting C (including output head and attention) (§3.1, §3.2)
- [ ] Lock primary cross-entropy loss, fixed evaluation corpus, token accounting, and progress-proportional evaluation intervals (§3.3)
- [ ] Enforce uniform architectural primitives, aspect-ratio progression, and MoE routing/balancing rules (§4.2, §4.4)
- [ ] Run the target numerical precision scheme; use byte-normalized BPB when comparing tokenizers (§4.5)

Hyperparameter Search Phase

- [ ] Tune small models onto the interior of the Fully-Tuned Frontier for Objectives 2–4; follow prescribed recipe rules for Objective 1 (§6.1)
- [ ] Re-verify hyperparameter transfer at small scale after changing optimizers or depth scaling rules (§6.2)
- [ ] Measure batch size in tokens, initialize with $B_{opt} \propto D^{0.4\sim0.57}$, and test the impact of batch size ramps (§6.3)
- [ ] Check weight decay shifts or calibrate via timescale τ when spanning wide TPP ranges (§6.4)
- [ ] Re-evaluate winning hyperparameter configs with a fresh random seed to remove selection bias, and never prune trials before annealing completes (§6.7)

Curve Fitting Phase

- [ ] Confirm output-head FLOP accounting, proportional warmup, and small-model tuning completeness before fitting (§8)
- [ ] Compare Skaling and Chinchilla forms on validation slices (§8.1); include effective data volume decay under multi-epoch repetition (§7.3)
- [ ] Lock the early-checkpoint truncation rule before fitting (§5.4)
- [ ] Use multi-start robust regression, block-bootstrap over independent runs or shared-trunk clusters, and propagate uncertainty to the final decision (§8.3)
- [ ] Inspect two-dimensional residual plots for saddle curvature or monotonic drift, separating seed noise from functional-form error (§8.4)

Validation and Launch Phase

- [ ] Validate extrapolation on out-of-sample holdouts against pre-committed thresholds without tuning on the holdout (§10.1)
- [ ] Validate task-level NLL or clustering predictions on holdouts before relying on downstream forecasts (§9)
- [ ] Validate the combined stack of all winning changes end-to-end (§10.2)
- [ ] Run $2\times$ / $4\times$ high-LR stress tests and monitor long-horizon stability metrics after architecture or optimizer changes (§10.3)
- [ ] Verify single-step update parity, short-trajectory convergence, and stateful checkpoint resumption between research and production stacks (§10.4)

- [ ] Combine measured cluster throughput with serving latency and memory constraints when sizing the production model (§10.5)
- [ ] Run an intermediate-scale dress rehearsal on the production stack when extrapolating across large compute gaps (§10.6)
- [ ] Archive full run provenance, versioned fitting scripts, and golden regression benchmarks (§12)

Appendix A. Public Ladder Configurations

A.1 Public Scale Configurations

Model scales and training volumes used in major public Scaling Law studies (parameter conventions follow each source):

Source	Model Parameter Range	Training Volume / Compute Range	Reference
OpenAI	Multiple sizes, up to 1.5B (non-embedding)	22M–23B tokens	Kaplan et al., 2020
DeepMind Chinchilla	70M–16B (400+ models)	5B–500B tokens	Hoffmann et al., 2022
StepFun	Multiple sizes (3,700+ models)	~100T tokens cumulative across all runs	Step Law (Li et al., 2025)
Gadre et al.	11M–6.9B (104 models)	Up to 32× Chinchilla ratio	Gadre et al., 2024
Fantastic Optimizers	0.1B–1.2B (4 sizes)	1×–8× Chinchilla ratio	Wen et al., 2025
Delphi	Sweeps size and tokens by compute budget, max holdout 25B	Fit 3×10^{18}–3×10^{20} FLOPs, holdout up to 10^{23} FLOPs	Marin, 2026
Llama 3	40M–16B	Fit 6×10^{18}–10^{22} FLOPs, target 3.8×10^{25} FLOPs	Llama 3, 2024 §3.2.1

Training volumes of representative open-weight release models (for target production TPP comparison):

Source	Released Model Sizes	Cumulative Training Volume	Reference
NVIDIA Nemotron	15B, 340B	8T, 9T tokens	15B report; 340B report
OLMo	1B, 7B, 13B, 32B	OLMo 1: 2T–2.46T; OLMo 2: multi-stage budgets up to 5T+	OLMo, 2024; OLMo 2, 2025
Llama 3	8B, 70B, 405B	405B: 15.6T tokens (near compute-optimal); 8B/70B heavily over-trained	Llama 3, 2024 §1, §3.4

A.2 Fantastic Optimizers Ladder

Fantastic Optimizers (Wen et al., 2025) Table 2–3 defines a dense benchmark tailored for fair optimizer comparisons and rigorous hyperparameter search: built on the Llama 2 architecture, all four sizes fix depth at 32 layers, MHA, and sequence length 4096, scaling parameter count purely via `hidden_dim`.

Size	hidden_dim	inter_dim	heads	Horizon Tiers
130M	512	2,048	8	1×–8× Chinchilla
300M	768	3,072	12	Same as above
520M	1,024	4,096	16	Same as above
1.2B	1,536	6,144	24	Same as above

The corpus mixes DCLM-baseline, StarCoder V2 Data, and ProofPile 2. Each (size, horizon) grid point undergoes independent hyperparameter tuning (for example, Table 3 lists an AdamW optimum of Peak LR 8e-3, WD 0.1, warmup 2000 steps, and BSZ 128 sequences or ~0.5M tokens at one tier, shifting to WD 0.2 and BSZ 256 at 520M/1×).

A.3 Delphi Ladder

Delphi (Marin, 2026) exemplifies a dense Ladder built for end-to-end loss extrapolation across wide compute spans: based on the Qwen 3 decoder-only architecture (MLP ratio 4, sequence length 4096) and later extended to MoE (535B-A23B).

All runs share the AdamH optimizer, WSD schedule (10% warmup, 20% linear decay to 0), FP32 master weights with BF16 compute, and FSDP across a mixture of Nemotron-CC, StarCoderData, and ProofPile 2. Delphi sweeps IsoFLOP slices from 3×10^{18} to 3×10^{20} FLOPs and fits a power law through the 7 slice minima, setting hyperparameters via prescribed scaling formulas rather than manual per-point grids, and validates extrapolation across stepped holdouts from 10^{21} to 10^{23} FLOPs (up to 25B parameters, spanning $3\times$–$333\times$ extrapolation).

A.4 Selection Recommendations

- For hyperparameter scaling laws and fair algorithm comparisons: follow the $(N \times \mathrm{TPP})$ Cartesian grid of Fantastic Optimizers, investing compute to tune every small grid point onto the Fully-Tuned Frontier.
- For end-to-end compute allocation and large-scale loss forecasting: follow Delphi's IsoFLOP layout and formula-driven hyperparameters, using stepped holdouts to catch late-stage divergence.
- For production MoE targets: both public templates above are dense starting points; factor MoE experiments along the orthogonal axes of §5.6 and align GQA grouping and aspect-ratio progression with the target architecture (§4.2).

References

Grouped by topic, with back-links to the corresponding sections in the main text.

Scaling Law Foundations, Functional Forms, and Fitting

1. Scaling Laws for Neural Language Models — Kaplan et al., OpenAI, 2020. arXiv:2001.08361
   Empirical power laws across 7 orders of magnitude and non-embedding parameter conventions (§3.1, §4.3)
2. Training Compute-Optimal Large Language Models (Chinchilla) — Hoffmann et al., DeepMind, 2022. arXiv:2203.15556
   IsoFLOP methodology, proportional (N, D) scaling, and additive power-law parameterization (§3.1, §5.3, §8.1)
3. Language models scale reliably with over-training and on downstream tasks — Gadre et al., 2024. arXiv:2403.08540
   Power-law extrapolation up to 32× Chinchilla over-training and downstream error mappings (§5.3, §9.1)
4. Skaling: Chinchilla's Exponents Meet Kaplan's Coupling — Videau et al., FAIR at Meta, 2026. arXiv:2608.07222
   Outer coupling exponent k eliminating corner saddle residuals and enabling L-shape sparse grids (§8.1)
5. Predictable Scale: Part II, Farseer: A Refined Scaling Law in Large Language Models — Li et al., StepFun & Fudan, NeurIPS 2025. arXiv:2506.10972
   Nine-parameter coupled surface and ablation on excluding embedding parameters (§3.1, §8.1)
6. Scaling Law with Learning Rate Annealing — Tissue et al., 2024. arXiv:2408.11029
   Full-trajectory loss curve modeling via cumulative learning rate area and annealing kernels (§6.5, §8.2)
7. A Hitchhiker's Guide to Scaling Law Estimation — Choshen et al., MIT/IBM, ICML 2025. arXiv:2410.11840
   Empirical study of early checkpoint truncation, size spans, and seed variance (§2.3, §5.2, §5.4, §5.5)
8. Resolving Discrepancies in Compute-Optimal Scaling of Language Models — Porian et al., NeurIPS 2024. arXiv:2406.19146
   How output-head FLOPs, fixed warmup steps, and under-tuned hyperparameters distort compute-optimal exponents (§3.1, §4.1, §8)
9. Chinchilla Scaling: A Replication Attempt — Besiroglu et al., Epoch AI, 2024. arXiv:2404.10102
   Replication of Chinchilla's parametric fit highlighting solver and confidence-interval sensitivity (§8.3)
10. (Mis)Fitting: A Survey of Scaling Laws — Li et al., 2025. arXiv:2502.18969
    Survey of how omitted fitting objectives and numerical settings impact reproducibility (§8.3)
11. Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws — Sardana et al., 2024. arXiv:2401.00448

Inference-aware compute allocation and extreme over-training experiments up to 10,000 TPP (§5.3, §10.5)

12. Gemstones: A Model Suite for Multi-Faceted Scaling Laws — McLeish et al., 2025. arXiv:2502.06857
    Aspect-ratio effects on downstream tasks and sensitivity of compute-optimal fits to grid point selection (§2.3, §4.3)
13. Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies — Tao et al., NeurIPS 2024. arXiv:2407.13623
    Power-law growth of optimal vocabulary size with training compute (§4.5)
14. Scaling Laws for Precision — Kumar et al., 2024. arXiv:2411.04330
    Joint scaling laws for training precision and post-training quantization (§4.5)

Hyperparameter Scaling Laws and Optimizers

15. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism — DeepSeek, 2024. arXiv:2401.02954
    Compute-based hyperparameter power laws and corpus-quality shifts in optimal N/D (§6.3, §7.1)
16. Predictable Scale: Part I, Step Law — Optimal Hyperparameter Scaling Law in Large Language Model Pre-training — Li et al., StepFun, 2025. arXiv:2503.04715v3
    Bivariate $\eta_{opt}(N, D)$ and $B_{opt}(D)$ scaling laws calibrated across 3,700+ models (§6.1, §6.3)
17. Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training — Bergsma et al., Cerebras, 2025. arXiv:2505.13738
    $B_{opt} \propto D^{0.4}$, $B_{crit} \propto D_{min}^{0.5}$ hyperbolas, and EMA timescale τ coupling (§6.2, §6.3, §6.4)
18. Scaling Optimal LR Across Token Horizons — Bjorck et al., Microsoft, 2024. arXiv:2409.19913
    Power-law decay of optimal peak LR with token horizon at fixed model size and batch size (§6.3)
19. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (μ-Transfer) — Yang et al., Microsoft, 2022. arXiv:2203.03466
    Zero-shot learning rate transfer across width under μP parameterization (§6.2)
20. Tensor Programs VI: Feature Learning in Infinite-Depth Neural Networks — Yang et al., 2023. arXiv:2310.02244
    Single-layer Depth-μP and theoretical limits of infinite-depth parameterizations in multi-layer blocks (§6.2)
21. Depthwise Hyperparameter Transfer in Residual Networks: Dynamics and Scaling Limit — Bordelon et al., 2023. arXiv:2309.16620
    $1/\sqrt{\text{depth}}$ residual scaling for width-and-depth transfer in ResNet and ViT (§6.2)
22. Muon is Scalable for LLM Training — Liu et al., Moonshot AI, 2025. arXiv:2502.16982
    Update RMS alignment and weight decay enabling Muon to reuse tuned AdamW hyperparameters (§6.2)
23. Fantastic Pretraining Optimizers and Where to Find Them — Wen et al., Stanford, 2025. arXiv:2509.02046
    Controlled optimizer tuning benchmark, scale decay of speedups, and annealing rank inversions (§5.3, §6.8, Appendix A.2)
24. Weight Decay Improves Language Model Plasticity — Han et al., 2026. arXiv:2602.11137
    Decrease of pretraining-optimal WD with TPP and plasticity benefits of higher WD (§6.4)
25. Small-Scale Experiments: Are We There Yet? — Lourie et al., NYU & Meta, 2026. arXiv:2608.11859
    Small-model hyperparameter sensitivity, the Fully-Tuned Frontier, and fit/validation/test size splits (§5.4, §6.1)
26. How to Allocate Your Tokens? Scaling Laws with Training Steps and Batch Size — Schaipp, Inria, 2026. arXiv:2607.01487
    Characterization of the ~4× near-optimal batch size window within 5% compute overhead (§6.3)
27. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation — Cawley & Talbot, JMLR 2010. JMLR 11
    Analysis of selection bias when choosing the best configuration across finite noisy trials (§6.7)

MoE

28. Scaling Laws for Fine-Grained Mixture of Experts — Krajewski et al., 2024. arXiv:2402.07871
    MoE scaling laws incorporating expert granularity as an independent dimension (§4.4, §5.6, §8.5)
29. Parameters vs FLOPs: Scaling Laws for Optimal Sparsity for Mixture-of-Experts Language Models — Abnar et al., Apple, 2025. arXiv:2501.12370
    Optimal sparsity laws under fixed FLOPs vs. fixed total parameters and downstream transfer (§4.4)

30. Joint MoE Scaling Laws: Mixture of Experts Can Be Memory Efficient — Ludziejewski et al., 2025. arXiv:2502.05172
Joint scaling of expert count, active parameters, and training tokens under memory constraints (§4.4, §8.5)

Data Construction, Mixture, and Repetition

31. Scaling Data-Constrained Language Models — Muennighoff et al., 2023. arXiv:2305.16264
Exponentially decaying effective data volume formula for multi-epoch repetition (§7.3)
32. To Repeat or Not To Repeat: Insights from Scaling LLM under Token-Crisis — Xue et al., 2023. arXiv:2305.13230
Scaling of repetition overfitting with parameter count N and mitigation via dropout (§7.3)
33. Prescriptive Scaling Laws for Data Constrained Training — Lovelace et al., 2026. arXiv:2605.01640
Joint overfitting dynamics and compute allocation across parameters, unique tokens, and epochs (§7.3)
34. Larger Datasets Can Be Repeated More: A Theoretical Analysis of Multi-Epoch Scaling in Linear Regression — Yan et al., 2025. arXiv:2511.13421
Logarithmic scaling of optimal repetition epochs with dataset size in linear models and LLMs (§7.3)
35. UniMax: Fairer and More Effective Language Sampling for Large-Scale Multilingual Pretraining — Chung et al., ICLR 2023. arXiv:2304.09151
Epoch-capped sampling baseline for multi-source pretraining (§7.2)
36. Olmix: A Framework for Data Mixing Throughout LM Development — Chen et al., Allen Institute, ICLR 2026. arXiv:2602.12237
Proxy sizing, per-task log-linear regression, repetition caps, and incremental mixture reuse (§7.2)
37. Scaling Laws for Mixture Pretraining Under Data Constraints — Sedova et al., Apple, 2026. arXiv:2605.12715
High repetition tolerance of scarce domains in general mixtures and repetition-aware mixture laws (§7.3)
38. Decouple Searching from Training: Scaling Data Mixing via Model Merging for Large Language Model Pre-training (DeMix) — Li et al., 2026. arXiv:2602.00747
Evaluating candidate data mixtures via weighted merging of single-domain component models (§7.2)
39. Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset — Su et al., NVIDIA, 2024. arXiv:2412.02595
Refined Common Crawl and synthetic rewrites, a primary cross-source overlap in global deduplication (§7.3)

LR Schedule and Training Wrap-up

40. Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape Perspective — Wen et al., 2024. arXiv:2410.05192
River-valley loss-landscape geometry explaining the stable and decay phases of WSD (§6.5)
41. Scaling and Transferability of Annealing Strategies in Large Language Model Training — Wang et al., 2025. arXiv:2512.13705
Cross-scale transferability of LR annealing ratios and decay shapes (§6.5)
42. Model Merging in Pre-training of Large Language Models — ByteDance Seed, 2025. arXiv:2505.12082
Approximating annealed endpoint performance via WSD stable-phase checkpoint merging (PMA) (§6.9)
43. Model Soups: Averaging Weights of Multiple Fine-tuned Models — Wortsman et al., 2022. arXiv:2203.05482
Weight averaging across branches independently fine-tuned from a shared checkpoint (§6.9)
44. Stop Wasting My Time! Saving Days of ImageNet and BERT Training with Latest Weight Averaging (LAWA) — Kaddour, 2022. arXiv:2209.14981
Sliding-window weight averaging along the tail of a single trajectory (§6.9)
45. Early Weight Averaging meets High Learning Rates for LLM Pre-training — Sanyal et al., COLM 2024. arXiv:2306.03241
Early sliding weight averaging during high-LR pretraining (§6.9)

Downstream Task Prediction and Evaluation

46. Unveiling Downstream Performance Scaling of LLMs: A Clustering-Based Perspective (COD) — Xu et al., ICLR 2026, v4 (2026-03-09). arXiv:2502.17262v4
Four-stage difficulty-clustering framework for downstream benchmark extrapolation (§9.1, §9.2)

47. GPT-4 Technical Report — OpenAI, 2023. arXiv:2303.08774
Difficulty bucketing of HumanEval items by small-model pass rates (§9.1)
48. Establishing Task Scaling Laws via Compute-Efficient Model Ladders (OLMo Task Ladder) — Bhagia et al., Allen Institute, 2024. arXiv:2412.04403
Two-stage $(N, D) \to \text{Task NLL} \to \text{Accuracy}$ prediction pipeline (§9.1)
49. Why Has Predicting Downstream Capabilities of Frontier AI Models with Scale Remained Elusive? — Schaeffer et al., 2024. arXiv:2406.04391
How distractor probability mass and argmax thresholding degrade downstream predictability (§9.3)
50. RULER: What's the Real Context Size of Your Long-Context Language Models? — Hsieh et al., NVIDIA, 2024. arXiv:2404.06654
Multi-hop tracing and aggregation benchmark for evaluating effective context length (§11.2)

Model Technical Reports and Ladder Examples

51. Delphi: An Open Scaling Suite from 3e18 to 1e23 FLOPs — Marin Team, 2026. openathena.ai/blog/delphi
IsoFLOP sweeps, formula-driven hyperparameters, stepped holdouts, and optimum bootstrap (§1.1, §2.1, §5.2, §8.4, §9.1, §10.1, Appendix A.3)
52. The Llama 3 Herd of Models — Meta, 2024. arXiv:2407.21783
405B IsoFLOP sizing, small-model over-training, two-stage task prediction, BSZ ramp, and fault recovery (§5.2, §5.3, §6.3, §9.1, §10.4)
53. DeepSeek-V3 Technical Report — DeepSeek, 2024. arXiv:2412.19437
Auxiliary-loss-free bias load balancing, zero-drop routing, and FP8 mixed-precision design (§4.4, §10.4)
54. Nemotron-4 15B Technical Report — NVIDIA, 2024. arXiv:2402.16819
Batch size ramp-up and late-stage continued-training decay (§6.3, §6.9)
55. Nemotron-4 340B Technical Report — NVIDIA, 2024. arXiv:2406.11704
340B training configuration and token horizon reference (Appendix A.1)
56. OLMo: Accelerating the Science of Language Models — Groeneveld et al., Allen Institute, 2024. arXiv:2402.00838
Open pretraining suite, corpus, and intermediate checkpoint reference (Appendix A.1)
57. 2 OLMo 2 Furious — OLMo Team, Allen Institute, 2025. arXiv:2501.00656
Two-stage curriculum, micro-annealing data probes, and model souping (§6.9, §7.1)
58. OLMo 3 — Allen Institute, 2025. arXiv:2512.13961
BPB proxies, Olmix mixture iterations, quality-aware upsampling, and post-training checks (§2.2, §6.4, §7.2, §7.3, §10.2, §11.1)
59. Qwen3 Technical Report — Qwen Team, 2025. arXiv:2505.09388
Stage-wise hyperparameter prediction and fine-grained instance-attribute data mixing (§6.4, §7.2, §11.1)
60. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability — Qwen Team, 2026. arXiv:2608.30320
Wide hyperparameter plateau at scale, Muon hyperparameter shifts, post-training checks, and high-LR stress tests (§2.2, §6.1, §6.2, §10.3)
61. Kimi K2: Open Agentic Intelligence — Moonshot AI, 2025. arXiv:2507.20534
MoE sparsity scaling law at fixed active scale and multi-pass synthetic rewriting ablations (§5.6, §6.4, §7.3)
62. Kimi K2.5: Visual Agentic Intelligence — Moonshot AI, 2026. arXiv:2602.02276
Per-source maximum epoch caps during continued joint pretraining (§7.3)
63. Kimi K3: Open Frontier Intelligence — Moonshot AI, 2026. arXiv:2607.24653
Full scaling-law rebuild after recipe updates, tuned cosine vs. WSD comparison, and rewriting reuse (§6.8, §7.1, §7.3)
64. Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning — NVIDIA, 2026. arXiv:2604.12374
Two-stage 25T-token mixture shifting from 80% diversity to 20% high-quality data (§7.2)
65. Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning — NVIDIA, 2026. arXiv:2606.15007
Late-training divergence fixes via FP32 output gradients and early annealing, plus legal synthetic data ablations (§7.1, §10.3, §10.5)

66. Marin: MoE and Training Efficiency Follow-up — Marin Team, 2026. openathena.ai/blog/pretraining-speedup
    Theoretical vs. realized wall-clock efficiency and combined validation of multiple recipe changes (§10.5, §10.2)
67. Marin Data Pipeline — Marin Team, 2026. openathena.ai/blog/marin-data-pipeline-overview
    Global cross-source deduplication, proportional proxy pool downscaling, non-monotonic mixture regression, and cross-scale confirmation (§7.2, §7.3)
68. Phi-4 Technical Report — Microsoft, 2024. arXiv:2412.08905
    Multi-sample variance reduction on high-noise evaluation benchmarks (§2.2)
69. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models — Zhipu AI, 2025. arXiv:2508.06471
    Impact of model depth and attention head configuration on reasoning performance (§4.3)

Further Reading

Additional references on critical batch size and grouped-query attention:

70. Critical Batch Size Revisited: A Simple Empirical Approach to Large-Batch Language Model Training — Merrill et al., 2025. arXiv:2505.23971
71. An Empirical Model of Large-Batch Training — McCandlish et al., 2018. arXiv:1812.06162
72. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints — Ainslie et al., Google, 2023. arXiv:2305.13245
73. Cost-Optimal Grouped-Query Attention for Long-Context Modeling — Chen et al., 2025. arXiv:2503.09579